LEARN · VERIFY — OFFLINE STUDY BOOKLET

Stability Antipatterns & Stability Patterns

How a small fault becomes a total outage — and the twelve moves that stop it.

1
Where cracks start
Every integration point, and the two speeds of network failure.

2
How cracks spread
Blocked threads, chain reactions, cascading failures, slow responses.

3
Load turns on you
Users, scale, unbalanced capacity, self-denial, dogpiles, automation.

4
The counter-moves
Twelve stability patterns and when each is the wrong one.

Print double-sided, short-edge bind, A4 · 100% scale (no “fit to page”).

Answer key starts at Section 9 — keep a sheet of paper over it.

How to use this booklet

The loop, per section

1. Read the **plain version** box first. If it doesn't make sense, the rest won't either.
2. Read the teaching. Say each boundary out loud: “X is *this*, its look-alike Y is *that*.”
3. Cover the page. Write the answer to **Q1** in the blank lines — in your own words, in full sentences. **Then** check the key.
4. Score yourself on completeness. Then do **Q2** and note whether it beat Q1.
5. Only then move on.

> **THE RULE THAT DOES MOST OF THE WORK**
>
> If you find yourself thinking “yes, I know this” — that is *recognition*, and it is not learning. The only evidence that counts is a written answer produced with the page covered.

The spacing schedule

Review at **day 1, day 3, day 7, day 16, day 35**. On a pass, jump to the next interval. On a fail, reset that item to 1 day. Use the grid at the back; one row per section.

What's in here

> **ONE WARNING BEFORE YOU START**
>
> “Don't make the mistake of assuming that a system that includes more of these patterns is superior to one with fewer of them. **‘Count of patterns applied’ is never a good quality metric.**” The goal is a *recovery-oriented mind-set*, not a checklist score.

SECTION 1

Learning goal & roadmap

LEARNING GOAL — WHAT YOU SHOULD BE ABLE TO DO WHEN YOU CLOSE THIS

Given a real outage description (“the process is up but nothing responds”, “one node died and ten minutes later all twelve were gone”, “the site was fine until marketing sent an email”), you can:

1. **Name the failure mode** in the industry's vocabulary, not in adjectives.
2. **Say which way it travelled** — sideways across peers, or upward across layers — and what carried it across the gap.
3. **Pick the pattern that stops it**, name the one it's most often confused with, and say why you didn't pick that one.
4. **State the price** of the pattern you chose. Every one of them costs something.

The core model in one paragraph

A fault is unavoidable — a bug, a leak, a failing switch, a marketing email. A fault becomes a **crack**. Cracks are harmless until something **propagates** them: a blocked thread, a redistributed load, a retry loop, an automated control plane. Two structural properties decide how far a crack runs: **tight coupling** (a failure here forces a change there) and **high interactive complexity** (nobody's mental model of the system is complete, so operators' instincts are wrong). Systems with both live at what James R. Chiles calls the **technology frontier**, where small events routinely become full-blown failures. Parts 1–3 catalogue what propagates cracks. Part 4 catalogues what stops them.

THE WHOLE THING FOR A SMART 15-YEAR-OLD

- Programs are constantly waiting on other programs across a network. Waiting is where they die.
- A thing that says “no” quickly is fine. A thing that says nothing at all is what kills you — because you sit there holding the phone.
- Enough of your workers stuck holding phones = you are down, even though nothing crashed.
- So: never wait forever, stop calling things that keep failing, and build walls so one flooded room doesn't sink the ship.

The four sections, and why they're in this order

SECTION	WHAT IT COVERS	WHY HERE
1 Where cracks start	Integration points; sockets and the TCP handshake; fast vs slow network failure; the 5 a.m. problem; HTTP-level and vendor-library hazards.	Every other failure here is downstream of a call across a wire. Start at the source.
2 How a crack spreads	Blocked Threads, Chain Reactions, Cascading Failures, Slow Responses, fan-in.	Faults are unavoidable; propagation is not. This is the section that actually decides your uptime.
3 Load turns on you	Users and sessions, Expensive/Unwanted/Malicious users, Scaling Effects, Unbalanced Capacities, Self-Denial Attacks, Dogpile, Force Multiplier, Unbounded Result Sets.	These are the *triggers* — the things that light the fault that the machinery in §2 then spreads.

4 The counter-moves	Timeouts, Circuit Breaker, Bulkheads, Steady State, Fail Fast, Let It Crash, Handshaking, Test Harnesses, Decoupling Middleware, Shed Load, Back Pressure, Governor.	Each one is the mirror image of something in §1–§3. Learn them *as answers*, not as a list.

> *“Everything breaks. Faults are unavoidable. Don't pretend you can eliminate every possible source of them… Assume the worst. Faults will happen. We need to examine what happens after the fault creeps in.”*

1

PART 1 OF 4

Where cracks start: every integration point

Sockets, handshakes, and the difference between failing fast and failing silently.

TENTH-GRADER VERSION

- Your program talks to other programs over a wire. Every one of those conversations can go wrong.
- The worst kind of "wrong" isn't an error message. It's **silence** — your code just sits there waiting.
- A worker stuck waiting can't do anything else. Enough stuck workers and your whole system is effectively down, even though nothing crashed.
- So: never wait forever, and never trust what comes back.

Start with the phone call

- You ring a friend to ask one question. Exactly three things can happen:
  - **They answer.** Fine.
  - **The line is dead** and you hear a tone immediately. Annoying, but it costs you two seconds and you move on.
  - **It rings… and rings…** and you stand there with the phone to your ear for thirty minutes.
- The third one is the one that ruins your afternoon — **not because you got a worse answer, but because your ear was occupied**. You couldn't take any other calls.
- That ear is a request-handling thread. That's the whole of this part in one image.

NOW THE SAME THREE OUTCOMES IN NETWORK TERMS

PHONE CALL	WHAT THE NETWORK IS ACTUALLY DOING	COST TO YOU
Dead line, instant tone	Nobody listening on the port → remote stack replies with a TCP **reset**. Your code gets an exception or a bad return value.	**Milliseconds.** This is a *fast failure*, and it's the good case: your language has made it obvious that connections fail, so programmers actually handle it.
Ringing forever	Someone *is* listening, but the port's **listen queue** is full of half-open connections. Your `open()` is blocked inside the OS kernel.	**Minutes.** Connection timeouts vary by OS but are measured in minutes. "The listen queue is the worst place to be."
They pick up, say nothing	Connection established, request sent, server never gets around to responding. Your `read()` blocks — and the default in most stacks is **block forever**.	**Unbounded**, unless you set a socket timeout. And then you must be ready for the exception.

> *"Network failures can hit you in two ways: fast or slow… The blocked thread can't process other transactions, so overall capacity is reduced. If all threads end up getting blocked, then for all practical purposes, the server is down.* ***Clearly, a slow response is a lot worse than no response.****"*

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — THE THING IT GETS CONFUSED WITH
Integration point	Any place your system talks to another one: socket, database call, HTTP request, queue, vendor client library, named pipe.	Checkout calling a credit-card authoriser; an app server calling Oracle; a feed from the pricing system.	Not the same as a **dependency**. The dependency is the *thing* you rely on; the integration point is the *wire*. The wire is what fails, and it fails even when the thing at the other end is healthy.
Fast failure	The call comes back with an error almost immediately.	"Connection refused." A TCP reset. HTTP 500.	Not the same as **Fail Fast** (a pattern, §5). Fast failure is something that *happens to you*; Fail Fast is something *you do to your callers*.
Slow failure	The call neither succeeds nor errors for a long time; the thread is held.	A dropped ACK → the stack retransmits → the caller blocks for twenty to thirty minutes.	Not the same as a **slow response**. A slow response eventually arrives. A slow failure eventually throws — much later, having burnt capacity the whole time.
Three-way handshake	SYN → SYN/ACK → ACK. The ritual that makes discrete packets look like a continuous connection.	Under 10 ms if both machines are on the same switch.	Not the same as "the connection." A connection is a **logical construct** — state in memory at two endpoints. Nothing on the wire *is* the connection; there are only packets.
Listen queue	The kernel's waiting room, per port, for connections that have SYN'd but not been accepted.	A hammered service whose accept loop can't keep up.	A **full** listen queue is *good news* — further attempts are refused quickly. Being *in* the queue is the bad case. Fullness fails fast; membership fails slow.
Idle-connection reaping (the "5 a.m. problem")	A firewall silently forgets a connection that has been quiet too long, then drops its packets — while both endpoints still believe the connection is fine.	A LIFO connection pool reuses one connection overnight, leaving 39 idle past the firewall's one-hour timeout. At the morning ramp-up all 39 hang at once.	Not the same as a **half-open socket**. Half-open gives you a reset or an error. A reaped connection gives you *nothing* — no ICMP unreachable — so the stack just retransmits until `tcp_retries2` expires.
Vendor client-library risk	The vendor's client code is ordinary code: it blocks, it pools internally, it hides sockets from you.	A messaging library whose `messageReceived()` callback runs on an unknown thread already holding unknown locks — a deadlock minefield. Plugged callbacks block `send()`, which blocks request handlers.	Not the same as **the vendor's server**. The server may well be hardened by a thousand deployments; the client library rarely is, and you usually can't see its source.

The 5 a.m. problem, in full — because it teaches the method, not just the fact

- **Symptom:** thirty app-server instances all hang within a five-minute window at ~5 a.m., every day, right as traffic ramps up. Restarting clears it.
- **First evidence:** thread dumps show every request-handling thread blocked inside the JDBC driver, in low-level socket reads and writes.
- **The clue was an absence.** Packet capture on the database network showed *nothing at all* — like Sherlock Holmes' dog that didn't bark.
- **Mechanism:** a firewall is a specialised router with a finite table of established connections and a "last packet" timestamp per entry. Past its idle timeout, it drops the entry. TCP was never designed for a third party in the middle, so there's no way to tell the endpoints. The firewall was also configured to suppress ICMP (those double as attacker probes), so packets went on the floor **silently**.
- **Why it took so long to fail:** the stack sends, waits for an ACK, doesn't get one, retransmits — faithfully. Linux 2.6 with `tcp_retries2=15` ≈ **twenty minutes** before the socket library is told the connection is broken. The HP-UX boxes: **thirty minutes**. Reading could block *forever*.
- **Why it happened at 5 a.m. specifically:** the connection pool used **last-in, first-out**. Overnight, one connection served all the light traffic; the other thirty-nine sat idle well past the firewall's one-hour timeout. When traffic ramped, all thirty-nine locked instantly — and eventually the one good connection got checked out by a thread already blocked elsewhere. Total hang.
- **Fixes considered and rejected:** validation query on checkout (`SELECT SYSDATE FROM DUAL`) — hangs. Discard connections older than an hour — tearing down sends a packet, hangs. A "reaper" thread — hairy.
- **Fix used:** Oracle's **dead connection detection**. Its purpose is to notice crashed clients, but the periodic ping packet was what mattered: it refreshed the firewall's "last packet" clock and kept the entry alive.

> *"The main lesson here is that **not every problem can be solved at the level of abstraction where it manifests.** Sometimes the causes reverberate up and down the layers. You need to know how to drill through at least two layers of abstraction to find the 'reality' at that level."*

Above the socket: HTTP and the shape of a hostile response

Everything above still runs on sockets, so it inherits every failure above *plus* its own. Ways a provider can hurt an HTTP caller:

- Accepts the TCP connection but never responds to the request.
- Accepts the connection but never *reads* the request. A large body fills the provider's TCP window, which fills your send buffer, which blocks your socket write — **you can't even finish sending**.
- Returns a status you don't handle: "418 I'm a teapot", or more realistically "451 Resource censored".
- Returns a content type you didn't expect — a generic HTML 404 page instead of JSON. (Your ISP injecting an HTML page on a failed DNS lookup is a real instance of this.)
- Claims JSON and sends plain text, kernel binaries, or Weird Al Yankovic MP3s.

THE PRACTICAL RULE

Use a client library that gives you **fine-grained control over both connect and read timeouts** and over response handling. **Avoid libraries that map responses straight into domain objects.** Treat a response as *data* — maps and lists — until you have confirmed it meets your expectations. Only then turn it into objects.

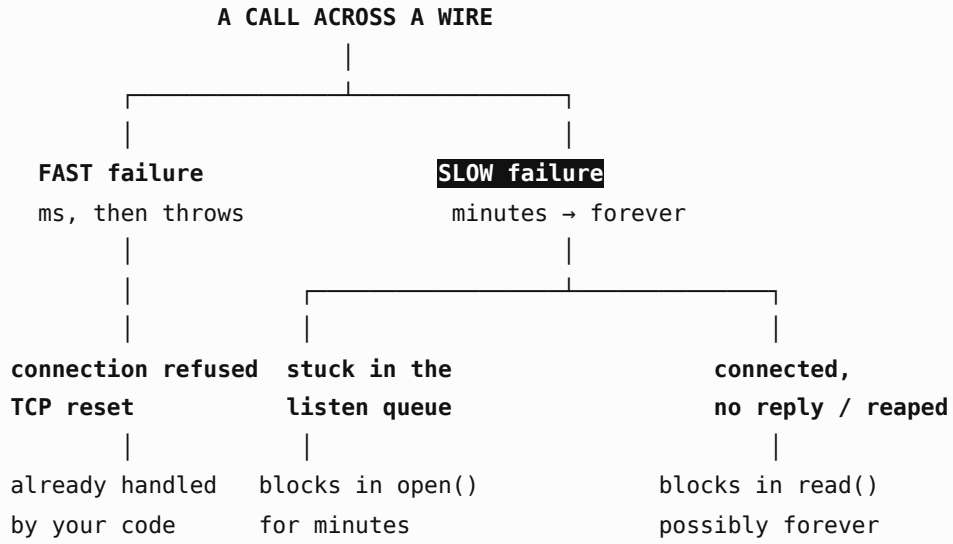
Trade-offs — nothing here is free

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Aggressive read timeout	A bounded blast radius; the thread comes back.	You will cut off some slow-but-valid work, and you need error-handling code at every call site.	Always. Derive the value from your own SLA, not from a round number.
Keep-alive ping on idle pooled connections	Survives firewalls and NAT devices that reap idle entries.	A trickle of extra traffic and one more thing to configure per data source.	Any long-lived pooled connection crossing a firewall.
Native / “thick” driver for better failover	Vendor failover features, sometimes better throughput.	Native code means a memory failure can **crash the runtime** instead of raising a catchable error.	When you actually need the failover feature and have tested the crash path.
Response → domain object automatically	Less code, faster to write.	A malformed or hostile response becomes an exception deep inside your object graph, or worse, a valid-looking object.	Only for providers you control, with schema validation in front.
Decompile and patch a vendor library	Unblocks you now.	An unsupported build you must carry until the official patch lands.	Genuine emergency, with a ticket open against the vendor.

> **CONCEPT BOUNDARY — WHAT AN INTEGRATION-POINT FAILURE *IS NOT***
>
> **It is not** a nice error response delivered through the defined protocol. That's the comfortable case you already handle.
> **It is** a protocol violation, a hang, a slow response, a wrong content type, a truncated body, or a connection that silently stopped existing. Expect the ugly ones; the protocol won't help you.

Anchor sketch — the failure taxonomy at one integration point

Legend: boxed = a named node | **reversed = the node that matters most** | **Q = cover the page and answer this**



Q Why is the node the dangerous one when the fast branch is the one that *throws an error*?

Q Which of the three leaves gives you **no packets at all** to look at, and what does that absence tell you?

Q Name the one setting that converts the right-hand branch into the left-hand branch.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“The other service didn't answer and our code just sat there.”	“The integration point hung — the calling thread blocked on a socket read with no timeout, so we lost capacity rather than getting an error.”
“It broke instantly.”	“Fast failure — connection refused, TCP reset, a few milliseconds. That's the cheap case.”
“The firewall forgot about us.”	“The firewall reaped the idle entry from its connection table and dropped subsequent packets without an ICMP unreachable, so our stack retransmitted until `tcp_retries2` expired.”
“We had to go look at the actual network.”	“We dropped a layer of abstraction and took a packet capture — `tcpdump` in promiscuous mode, analysed off-box in Wireshark.”
“Their client library is sketchy.”	“The vendor client library does unsafe blocking with no configurable failure modes, and it hides the socket so we can't set our own timeouts.”
“Don't just parse it straight into our objects.”	“Treat the response as data until it's validated — deserialise into maps and lists, then extract. Cynical parsing.”

> *“Integration points are the number-one killer of systems. Every single one of those feeds presents a stability risk. Every socket, process, pipe, or remote procedure call can and will hang.”*

Retrieval — cover the page first

PART 1 · QUESTION 1

Your payments service calls an external fraud-scoring API. During an incident, monitoring shows CPU at 3%, memory flat, no errors in the application log — and yet every request-handling thread is occupied and the service returns nothing to anyone.

(a) Name the failure mode at the integration point. **(b)** Explain why the *absence* of errors and the *absence* of load is the strongest clue you have. **(c)** Name the two settings you would check first, and say which one you'd bet on.

PART 1 · QUESTION 2

A nightly job holds a pooled TCP connection to a partner's service. It runs fine all week. Every Monday at 06:10 it hangs for about thirty minutes, then throws. No code has changed. The partner insists their service was up the whole time — and their logs prove it.

(a) State your hypothesis in one sentence. **(b)** Name the single packet-level observation that would confirm it. **(c)** Give the cheapest fix and say why the two obvious fixes — validating the connection before use, or closing connections older than an hour — both fail.

2×2 — how to classify any integration-point failure in ten seconds

The two questions worth asking during an incident: **how fast did it fail?** and **did it fail inside the protocol or outside it?** The answers land you in one of four boxes, and each box has a different move.

COLUMNS: HOW THE FAILURE BEHAVED WITH RESPECT TO THE PROTOCOL →

	Inside the protocol a defined error you can read	**Outside the protocol** a violation, a hang, or garbage
FAST **milliseconds**	**The comfortable quadrant** Connection refused · TCP reset · HTTP 500 · a full listen queue rejecting quickly. *Move:* ordinary error handling. Already covered by your language's exception model, which is exactly why developers do handle it.	**Cynicism required** HTML 404 page where JSON was promised · content type lies · truncated body · “451 Resource censored”. *Move:* parse as data first, validate, then build objects. Never map straight into your domain.
SLOW **minutes → forever**	**Timeout territory** Sat in the listen queue · provider accepted and is taking its time · TCP window full so even your write blocks. *Move:* explicit connect *and* read timeouts, plus a checkout timeout on the pool behind it.	**The killer quadrant** The 5 a.m. problem. Firewall reaped the entry, packets dropped silently, no ICMP, endless retransmits. Both ends still “believe” they're connected. *Move:* timeouts and a keep-alive that refreshes the middlebox's clock — and a packet capture, because nothing at the application layer will show you this.

Read it as: the danger rises as you go down and to the right. Everything in the bottom-right is invisible from inside your application; you must drop a layer to see it.

CARRY-OUT RULE FROM THIS PART

Every wait must be bounded, and every response must be doubted. If you can only afford one change to a legacy integration, make it the read timeout — that single setting moves failures from the bottom row to the top row, where your code already knows what to do.

WHICH PATTERNS ANSWER THIS PART (FORWARD REFERENCE TO §5)

- **Timeouts** — stop waiting for an answer that isn't coming.
- **Circuit Breaker** — stop making the call at all once it's clearly sick.
- **Decoupling Middleware** — remove the synchronous wait entirely.
- **Handshaking** — ask whether the provider is ready before you commit a thread.
- **Test Harnesses** — a deliberately malicious fake provider, so you meet these failures before production does.

2

PART 2 OF 4

How a crack becomes an outage

Blocked Threads, Chain Reactions, Cascading Failures, Slow Responses — the propagation machinery.

TENTH-GRADER VERSION

- One broken thing almost never stays one broken thing.
- When a server dies, its work goes to its neighbours — which makes *them* more likely to die too.
- When something you depend on dies, everything of yours that was waiting on it dies with it.
- And slowness travels *upward* and multiplies, because impatient users hit Reload and add even more traffic.

Start with the bridge

- A four-lane bridge closes one lane. The other three now carry a third more traffic each. If they were already near capacity, the next lane buckles sooner, then the next — **sideways, within the bridge**.
- Meanwhile the roads *feeding* the bridge back up all the way into the city. The jam has jumped from one layer (the bridge) to another (the street grid) — **upward, across a boundary**.
- Those two directions are the two antipatterns everybody mixes up. Sideways among peers is a **chain reaction**. Upward across a layer boundary is a **cascading failure**.

THE ARITHMETIC THAT MAKES A CHAIN REACTION INEVITABLE

FARM SIZE	LOAD PER NODE, HEALTHY	AFTER ONE NODE DIES	INCREASE ON EACH SURVIVOR
8 servers	12.5%	14.3%	+1.8 points of total load — but **+15% on that server**
2 servers	50%	100%	**+100%** — the survivor's workload doubles

That's the whole mechanism: if the first node died of something *load-related* — a memory leak, an intermittent race condition — then every survivor just got closer to the same cliff. And they're running the same code, so they have the same defect.

> ***The search farm.*** *A dozen search engines behind a load balancer, half a million SKUs. The engine had a memory leak. Because the load balancer had shared traffic evenly all morning, they all approached death together. As each went dark its share went to the survivors, which then ran out of memory faster. Charting their "last response" timestamps showed the acceleration plainly: five or six minutes between the first crash and the second, three or four between the second and third, seconds between the last two. Losing the final search server locked up the entire front end — that part was the cascade.*

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY

Chain Reaction	Failure spreads **sideways within one horizontally-scaled layer**, because the dead node's load lands on its peers.	The search farm above. Also: a race condition that only trips under high load, so each new death makes the next one likelier.	vs **Cascading Failure**: same layer, peers, driven by *redistributed load* — not by a caller/provider relationship. The only real cure is fixing the underlying defect; bulkheads merely split one chain reaction into two slower ones.
Cascading Failure	A crack **jumps a layer boundary**, upward from provider to caller.	A database cluster goes dark → the connection pool drains → every caller thread blocks → the caller is down too.	vs **Chain Reaction**: crosses layers, and it needs a *mechanism* to jump the gap — usually blocked threads, sometimes an over-aggressive retry. "Cascading failures are the number-one crack accelerator."
Blocked Threads	The process is alive and the interpreter is running, but every thread that could serve a request is waiting on something that will never arrive.	See the cache story opposite. Also: any pool checkout without a timeout; any `synchronized` method that can make a remote call.	vs a **crash**: a crash is visible and a process monitor catches it. A hang looks perfectly healthy to a process monitor, a port monitor, and often to log scraping. Only a synthetic transaction from outside notices.
Slow Responses	Answers that do arrive, but too late to be useful.	Memory-leak-driven garbage collection showing as high CPU that isn't doing any transaction work. Or a hand-rolled socket protocol whose `read()` doesn't loop until the receive buffer is drained, stalling TCP.	vs **Fail Fast** (the pattern): slow response is the disease, Fail Fast is the treatment. And for a website, slow responses *generate* traffic — users hit Reload.
Fan-in	How many callers a service has.	An authentication service everything calls.	vs **fan-out**, which is how many services *you* call. High fan-in = wide blast radius; "crucial services with a high fan-in… spread their problems widely, so they are worth extra scrutiny."

The blocked-thread story worth memorising

- A retailer needed in-store availability. The inventory system was slow (seconds per call) and undersized, but results stayed valid ~15 minutes, and 25% of lookups were on the week's hot items. Obvious answer: cache them.
- The developer extended an existing `GlobalObjectCache` and overrode `get()` to do the remote call on a miss — a textbook read-through caching proxy, with timestamps for expiry. **Functionally it was a nice piece of work.**
- The parent's `get()` was declared `synchronized`: only one thread inside at a time. That was fine in tests, because every test returned quickly.
- In production the inventory back end got flooded and crashed. Now **one** thread sat inside `create()` waiting for a response that would never come — holding the monitor. Every other caller queued behind it. Every request-handling thread on the box, gone.
- Nobody would have said yes to "should the whole site go down if we can't check in-store availability?" — and nobody was asked.

> *"No one designed this failure mode into the combined system, **but no one designed it out either**."*

TWO TECHNICAL POINTS HIDING INSIDE THAT STORY

- **You cannot see blocking at the call site.** `globalObjectCache.get(key)` looks identical whether it's a hash lookup or a three-second network round trip. The interface said nothing about synchronisation either.
- **A subclass that adds `synchronized` violates the Liskov substitution principle** — any property true of type T should be true of its subtypes. Java and C# forbid the other LSP violations but allow this one. The underlying reason: **functional behaviour composes, but concurrency does not compose.** So you cannot transparently substitute the synchronised subclass for the superclass, even though the compiler says you can.

DESIGN RULES THAT FALL OUT OF IT

- If you're synchronising methods on domain objects, rethink the design. It breaks the moment you run on more than one server (in-memory coherence is meaningless when another box is changing the data), and it stops request threads scaling.
- Prefer **immutable** domain objects for query and rendering; change state by issuing a command object. That's Command Query Responsibility Separation, and it sidesteps a large class of concurrency problems.
- Don't roll your own connection pool. "It's always more difficult than you think to make a reliable, safe, high-performance connection pool" — and once you add metrics it stops being a fun exercise and becomes a grind.
- Third-party libraries that pool internally are a classic hiding place. If you can't set a timeout, you may have to wrap the call in your own worker-thread pool returning a future — but go far down that road and you've written a reactive wrapper around the whole client. *Here there be dragons.*

Why cascades happen even when everyone was careful

- **Resource pools are the usual gap-jumper.** A failure in a lower layer drains the pool; threads holding connections block forever; every other thread blocks waiting for a connection. Safe pools always cap how long a thread may wait to check one out.
- **Integration points without timeouts are a surefire way to create cascading failures.** That sentence is close to a law.
- **The reverse also happens — an over-aggressive caller.** One system got a quick error from a lower layer, and because of a historical race condition the upstream developer had decided to retry on that error. The lower layer couldn't distinguish transient errors from serious ones. When the lower layer developed a real problem (packet loss from a

failed switch), the caller hammered it harder and harder until it was spending 100% of CPU making failing calls and logging them.

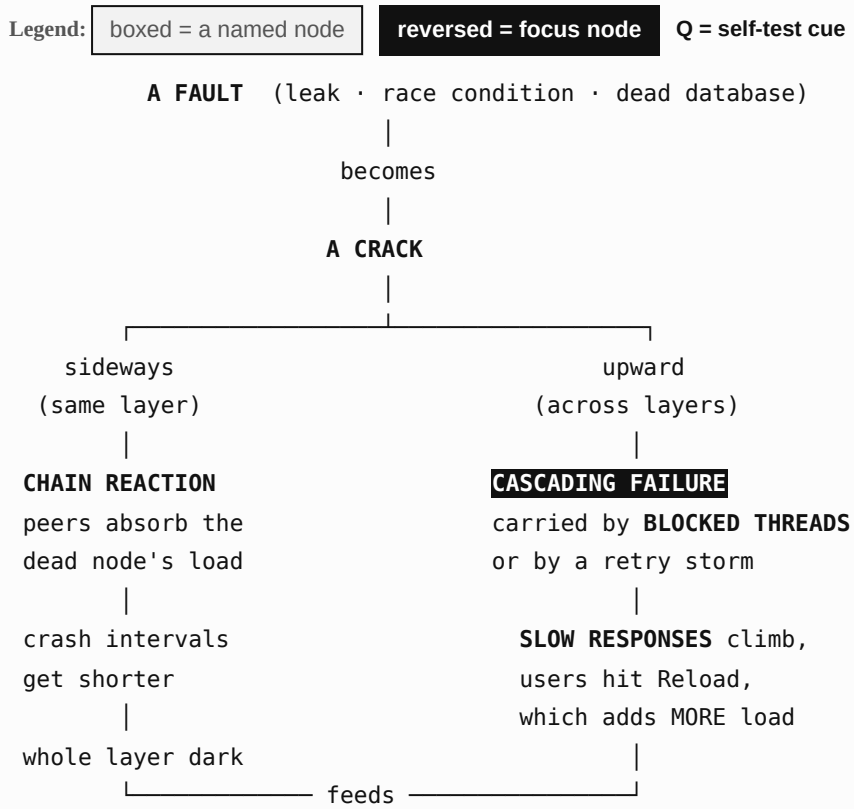
- **Speculative retries** jump the gap the same way: a provider slowdown makes the caller fire *more* hedged requests, tying up more of the caller's threads exactly when the provider can least afford the load.

Detection — the part most teams get wrong

- Internal monitors (process monitoring, port checks, log scraping) all report “healthy” during a total hang. The process *is* running.
- Supplement them with **external monitoring**: a mock client *outside your data centre* running synthetic transactions on a schedule. It sees what a user sees.
- Business-level counters catch it earliest of all: “successful logins”, “failed credit cards”. A counter going flat is visible long before an alert threshold fires.
- Stop arguing about “crashed” versus “hung”. Only one variable is observable to anyone who matters: **is it processing transactions?** Or in the sponsor's phrasing, “is it generating revenue?”

> **CONCEPT BOUNDARY — THREE FAILURES PEOPLE CALL BY EACH OTHER'S NAMES**
>
ANTIPATTERN	DIRECTION	CARRIED BY	THE DEFENCE
Blocked Threads	Inside one process	A monitor, a pool, or a socket read with no bound	Timeouts everywhere; proven concurrency primitives
Chain Reaction	Sideways, peer to peer	Redistributed load onto identical code with the same defect	Fix the defect. Bulkheads to split it. Autoscaling with health checks, if the scaler is faster than the reaction
Cascading Failure	Upward, layer to layer	Drained resource pools; retry storms; speculative retries	Circuit Breaker and Timeouts. “Preventing cascading failures is the very key to resilience.”

Anchor sketch — one crack, two directions



Q Why does the diagram loop back on itself at the bottom?

Q A chain reaction takes out a whole layer. What must the layer *above* have in place so this stays one outage instead of two?

Q Name the mechanism that most often carries a crack across a layer boundary — and one that carries it in the opposite direction from what you'd expect.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“One box died and then they all died, faster and faster.”	“A chain reaction across the horizontally-scaled layer — a load-related defect, so each death raised the load and the failure probability on the survivors.”
“The database went down and took the app with it.”	“A cascading failure. The crack jumped the layer boundary through an exhausted connection pool — threads blocked on checkout with no timeout.”
“The server's up but nothing works.”	“The process is running but every request-handling thread is blocked. Process and port monitoring show green; only synthetic transactions from outside the data centre catch it.”
“It got slower and slower and then it died.”	“Slow responses propagating upward as a gradual cascading failure, amplified by user reloads and by the caller's retries.”
“Loads of teams call this service.”	“It's a high fan-in service, so it warrants extra scrutiny — its blast radius is the whole enterprise.”
“We just retry when it errors.”	“Immediate retries against a sick provider are a gap-jumping mechanism; unless the provider distinguishes transient from serious faults, retrying converts its problem into our outage.”

Retrieval — cover the page first

PART 2 · QUESTION 1

Your recommendations service runs twelve instances behind a load balancer. At 14:02 one instance stops responding. By 14:09 all twelve are gone, and the gaps between failures got noticeably shorter each time. Upstream, the product-page service is also down — its own CPU and memory look normal, but its thread count is pinned at maximum.

(a) Name the two distinct antipatterns at work. **(b)** Say which one travelled sideways and which travelled upward, and what carried each. **(c)** Give the one pattern that would have stopped each, and say why swapping the two patterns would not have worked.

PART 2 · QUESTION 2

A team speeds up a slow partner API by subclassing an existing in-memory cache and overriding `get()` to fetch on a miss. The parent's `get()` is `synchronized`. Code review passes, load tests pass. In production, when the partner API stops responding, the **entire service** hangs — even though only 2% of traffic touches that feature.

(a) Explain the mechanism in one sentence. **(b)** Name the object-oriented principle violated and the one-line reason the compiler didn't stop it. **(c)** Give one code-level fix and one architectural fix, and say which one you'd ship tonight.

2×2 — locate any propagating failure

Two questions: **which way did it travel**, and **what carried it**. Where those meet tells you the antipattern's name and, more usefully, which side of the call you need to change.

COLUMNS: WHAT CARRIED THE FAILURE →

	Resource exhaustion threads, connections, memory	**Redistributed demand** load, retries, reloads
WITHIN one layer	**Blocked Threads** One monitor, one pool, one unbounded read. The process is alive and does nothing. *Fix on:* your own code. Timeouts on every wait, proven concurrency primitives, no `synchronized` around a remote call.	**Chain Reaction** A load-related defect in identical peers; the dead node's share lands on the survivors. *Fix on:* the defect itself. Then bulkheads to split the layer, and health-checked autoscaling that reacts faster than the reaction spreads.
ACROSS layers	**Cascading Failure** The lower layer dies; your pool drains; your threads block; you die too. *Fix on:* the **calling** side. Circuit Breaker to stop calling, Timeouts to come back from the calls you do make. This is the quadrant that decides your resilience.	**Slow Responses → retry storms** Latency climbs; callers retry, users reload, speculative retries multiply; the amplification is upward. *Fix on:* the **provider** side. Fail Fast inside your SLA, shed load, and never let a caller mistake “slow” for “try harder”.

Read it as: the left column is fixed by bounding waits; the right column is fixed by bounding demand. The bottom-left is the one that turns an incident into an outage.

CARRY-OUT RULE FROM THIS PART

Faults are unavoidable; propagation is optional. Ask of every dependency: “when this is dead, what of mine is holding a thread waiting for it?” Whatever that thing is, it is your gap-jumper — and it is where a timeout or a breaker belongs.

3

PART 3 OF 4

Load-shaped failure: demand, scale, and automation turning on you

Users · Scaling Effects · Unbalanced Capacities · Self-Denial · Dogpile · Force Multiplier · Unbounded Result Sets

TENTH-GRADER VERSION

- More users is the goal *and* the threat. Growth is what breaks the thing you built for growth.
- Some breakages only appear at production size — never on your laptop, never in the test environment.
- The worst traffic spikes come from inside your own building: a marketing email, a midnight cron job, a robot with bad information.
- Any number nobody bothered to put a limit on — sessions, rows, retries, servers — will one day be enormous.

Start with the supermarket

- One till serves the shoppers comfortably. Then a coupon goes viral and the store fills — **external surge**.
- Most shoppers grab one item and leave; a few do a full deli order with price checks and returns. The **expensive** ones are exactly the ones you want more of, because they're the ones who spend.
- The manager announces over the PA: “free samples at 5 p.m. sharp.” Everyone arrives in the same minute — **self-inflicted, synchronised**.
- And the single loyalty-card scanner that every till shares becomes the thing the whole store waits on — a **shared resource** bottleneck that got worse purely because you added tills.
- Four different failure shapes, one store. That's this part.

The demand side: your users

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY
Capacity	The maximum throughput you can sustain at acceptable performance under a given workload.	When a transaction takes too long, demand has exceeded capacity.	vs **performance**: performance is one transaction's speed; capacity is how many you hold at that speed. Passing internal hard limits creates cracks, and cracks propagate faster under stress.
Session memory & dead time	Every user costs memory, and the session lingers after the last request until it times out.	Keeping a whole search result set in session for pagination — it may never be read again, and it holds bytes for thirty minutes.	vs a **cache**: a cache is a bet you can lose cheaply. A session is memory you're obliged to hold. When memory runs short, you may not even be able to log the error, because logging needs memory.
Weak / soft references	A holder that keeps a payload only until the garbage collector needs the space.	`SoftReference` in Java, `System.WeakReference` in C#, `weakref` in Python. Wrap the expensive object, put the wrapper in the session.	vs an **eviction policy**: you don't decide — the collector does, and the only guarantee is that weakly-reachable objects go before an out-of-memory error. Callers must handle `null`. Best answer is still: keep it out of the session.

Off-heap / off-host memory	Move per-user state into another process or machine.	Memcached; Redis as a “data structure server” between cache and database.	A trade in the **memory hierarchy** — registers, cache, local memory, disk, tape. Networks are now fast enough that *someone else's memory* can beat your local disk. Local memory still beats remote memory. There is no one-size answer.
Socket & port limits	Port numbers are 16 bits, so one IP caps out.	IANA ephemeral range 49152–65535 ≈ 16,383 connections; stretch to 1024–65535 ≈ 64,511. A million concurrent connections needs ~16 virtual IPs bound to the interface, plus kernel buffer tuning.	vs **TIME_WAIT**, which is about sockets you already *closed*: a delay before reuse, protecting against **bogons** — stray late packets that could be accepted as legitimate data on a reused socket. Real on the open internet, largely irrelevant inside your own data centre, so you can turn the interval down.
Expensive users	The users who cost the most are the ones who buy.	Checkout is 4–5 pages and pulls in credit-card authorisation, tax calculation, address standardisation, inventory and shipping. Browsers are cacheable and touch almost no integration points.	There is **no defence**, and you shouldn't want one — they're the revenue. Instead *test* for them: if you expect a 2% conversion rate, load-test at 4, 6 or 10%. Testing at 100% is a fine stability test but a terrible capacity plan — build for that and you'll overspend tenfold.
Unwanted users	Not malicious, just destructive: broken proxies, home-brew bots, screen scrapers.	A misconfigured proxy replaying a user's last URL, accelerating to 4–5 requests a second, each without a session cookie so each creates a new session. Competitive-intelligence firms scraping your catalogue page by page.	The cheapest denial of service against a session-creating web app: request a deep link without cookies and **drop the socket immediately**. The web server never tells the app server the client left; the app server builds the session and the response anyway. 100 bytes of request, kilobytes of memory.
Malicious users	Deliberate attackers; overwhelmingly “script kiddies” by volume.	DDoS from a botnet, forcing you to saturate your own outbound bandwidth and rack up the bill. Session management is the most vulnerable point of a server-side web app.	vs the rare **advanced persistent threat**: if you're targeted by one you will almost certainly be breached — that's a security-consultant problem, not an architecture one. Run DDoS mitigation appliances in learning/baseline mode for at least a month first.

WHY SESSIONS KEEP SHOWING UP AS THE WEAK POINT

HTTP is stateless: each request emerges from the fog and disappears again. Cookies were grafted on to fix that, but real state in cookies costs bandwidth and CPU and is unsafe unencrypted — so cookies shrank to a tag, and the real state moved into server memory. **A session is a cache in memory keyed by a tag the client controls.** Anything that fails to send the tag manufactures a new one. That is why bots, scrapers and broken proxies all turn into memory exhaustion.

The structural side: scale, capacity ratios, and shared resources

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY
Scaling Effects	Any many-to-one or many-to-few relationship breaks when the "many" side grows.	The square–cube law: a bug's weight goes as $O(n^3)$, its leg strength as $O(n^2)$, so there are no elephant-sized spiders. A database that's fine with ten callers crashes at sixty.	Dev has one machine, test has two, production has *n*. That's precisely why these defects survive QA. **This type of defect cannot be tested out; it must be designed out.**
Point-to-point communication	Every instance talks directly to every other instance.	Connections grow as the square of instance count. Fine at two servers, painful at a hundred.	Distinguish point-to-point *inside* a service ($O(n^2)$, the problem) from the normal fan-in *between* services through a load balancer (a different case entirely). Replacements, cheapest first: UDP broadcast → multicast → publish/subscribe → message queues. "Do the simplest thing that will work."
Shared resource	A facility every member of a horizontally-scaled layer must use.	A distributed lock manager, a cluster manager, a cache-coherency manager.	Harmless when it is **redundant and non-exclusive** — then you scale the bottleneck. Dangerous when it's allocated **exclusively** per unit of work: contention scales with both transactions and clients; saturation → connection backlog → listen queue overflow → failed transactions → for cache managers, stale data or loss of integrity.
Shared-nothing architecture	Each server runs without knowing anything about any other. Capacity scales roughly linearly.	The hypothetical ideal for horizontal scaling; in reality there's always some coordination.	The catch is **failover**. Session failover needs the session to leave the original server — a backup server, a database table, a designated peer — and that means shared resources. Middle path: **reduce fan-in**, e.g. pair servers as each other's failover.
Unbalanced Capacities	One tier can flood another because their capacities were never matched.	Front end: 20 hosts, 75 instances, **3,000 threads**. Back end: 6 hosts, 6 instances, **450 threads**. Normally a tiny fraction of front-end threads call the back end — until marketing offers free installation for one day.	A **special case of Scaling Effects** where one side scaled far more than the other. Matching every service to the front end is a gross misuse of capital — the infrastructure would sit 99% idle for five years. So make both sides resilient instead: Circuit Breaker on the caller, Handshaking and Back Pressure on the provider, Bulkheads to reserve capacity for critical callers.

The self-inflicted side: your own organisation and your own robots

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY

Self-Denial Attack	The system — including its humans — conspires against itself.	An electronics retailer emailed the exact preorder date and time for the Xbox 360, with a deep link that bypassed the CDN so every image and stylesheet came from origin. “One minute before the appointed time, the entire site lit up like a nova, then went dark. It was gone in sixty seconds.”	vs an external flash mob: same shape, but **you knew in advance** and could have prepared. Machine-side version: a single rogue server damaging all the others through a shared lock manager. Defences: shared-nothing, no deep links in mass mail, send in waves, static landing zones, pre-scale *before* the campaign, fallback modes (pessimistic → optimistic locking), and above all keep the lines of communication open.
Fight Club bug	Increased front-end load causing exponentially increasing back-end work.	A popular item modified by mistake, so thousands of threads across hundreds of servers serialise on one write lock.	vs ordinary load growth, which is linear. Look for it wherever a shared resource is touched per request.
Dogpile	Demand that arrives all at once instead of spread out.	A fleet booting after a deploy, every instance with a cold cache hammering the database. Cron jobs all set to midnight. A config-management push. A server room whose breaker trips because every machine starts at once.	vs sustained overload: a dogpile is **transient but concentrated**, so it forces you to buy peak capacity you'd otherwise never need. Fixes: randomised clock slew, backoff with jitter (a fixed retry interval re-synchronises callers), random deltas on load-test think times. Think of pedestrians re-clumped by every traffic light.
Force Multiplier	Control-plane automation making a large mistake very quickly.	**Reddit, 11 Aug 2016**: admins stopped the autoscaler to upgrade ZooKeeper → the package manager noticed it was off and restarted it → it read half-migrated ZooKeeper data describing a much smaller fleet → it shut down most app and cache servers → manual restore brought instances back with **cold caches**, which dogpiled the database. ~90 minutes down, ~90 minutes degraded.	vs a plain bug: the automation was working **exactly as designed**. The failure was a divergence between its belief about desired state and reality. Same shape in a service-discovery node that gets network-partitioned: it can't tell “the universe vanished” from “I'm blindfolded”, and it gossips “everything crashed” to the healthy nodes.
Unbounded Result Set	The caller lets the other side decide how much data to send.	**Black Monday**: a table backing a database-implemented JMS should have held ~1,000 rows; it held over ten million. The app selected them all — with `select … for update`, so it held locks — ran out of memory, and because the type-2 JDBC driver ran *native* code, the JVM **crashed** rather than raising an out-of-memory error. The crash rolled back the transaction, released the lock, and the next instance walked off the same cliff. Over 75% of a hundred-instance farm down.	Not really a database problem — **it's a handshaking failure**. In any API the caller should state how much it's prepared to accept, the way TCP does with the window field. Beware relationships that accumulate without limit: orders→order lines, profiles→visits, anything with an audit trail. And beware distributions: social connections follow a **power law**, not a bell curve, so an entity with a million times the average *is guaranteed to exist*.

SAFEGUARDS FOR CONTROL-PLANE AUTOMATION — BORROWED FROM INDUSTRIAL ROBOT SAFETY

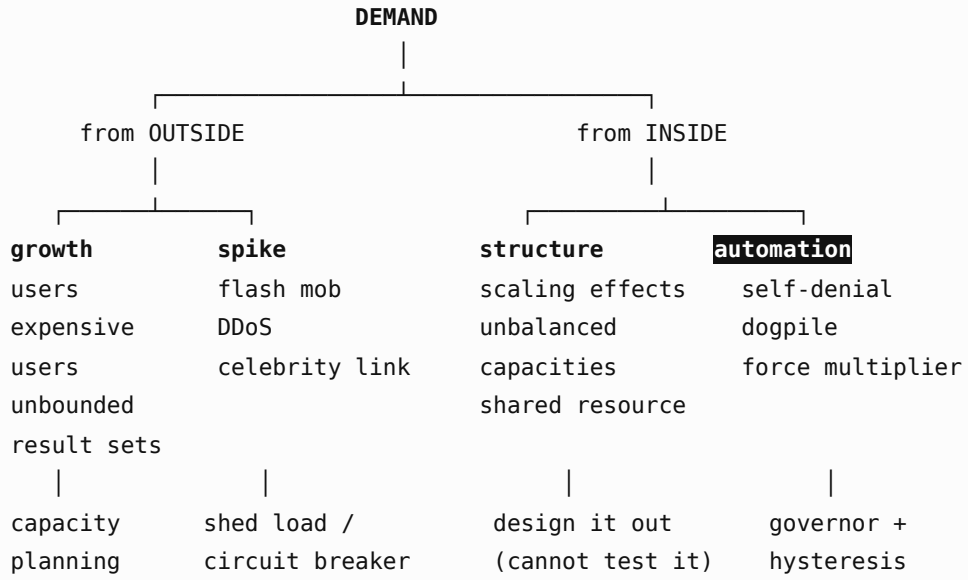
- If observations say more than **80% of the system is unavailable**, suspect the observer, not the system.
- **Apply hysteresis.** Start machines quickly; shut them down slowly. Starting is the safe direction.
- **Signal for confirmation** when the gap between expected and observed state is large — the software equivalent of the big yellow warning lamp.
- Resource-consuming systems should be **stateful enough to notice** they're about to spin up infinity instances.
- Build **deceleration zones** for momentum: if you sense load every second but a VM takes five minutes to boot, don't start 300 of them because the load persisted.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Sessions in Redis/Memcached instead of heap	Heap pressure gone; session survives an instance dying.	Network latency on every access; a new dependency that can itself fail.	Sessions are large, or you need failover without pinning users to instances.
Weak/soft references for expensive session data	Automatic relief exactly when memory is tight.	Indirection, null-handling in every accessor, non-deterministic behaviour that's miserable to test.	You genuinely can't keep it out of the session. Otherwise, just don't put it there.
Shared-nothing	Near-linear scaling, no coordination bottleneck.	You lose session failover and cache coherency, or must rebuild them.	Scaling matters more than transparent failover — or you can reduce fan-in instead of eliminating sharing.
Point-to-point → pub/sub	Kills $O(n^2)$ growth; late subscribers still get messages.	Serious infrastructure cost and operational surface.	Past roughly tens of nodes. Below that, point-to-point is genuinely fine — as long as it doesn't block when a peer dies.
Autoscaling	Absorbs surges without human intervention.	Minutes of lag; can push the overload onto a downstream platform service; buggy apps can autoscale you a spectacular bill.	Always with a **financial constraint** attached, and health checks on every group.
Blocking scrapers at the CDN or firewall	Reclaims real capacity — and it's a form of Circuit Breaker.	False positives on ISP super-proxies; scrapers rotate addresses and user agents; blocks must be expired or your firewall slows down.	You've identified them and can afford the maintenance. “Consider it pest control — once you stop, the infestation resumes.”
Pagination in the API contract	Ends the whole unbounded-result-set class of failure.	A contract change: first-item and count parameters in, approximate total out; callers must loop.	Always, for any query that isn't guaranteed to return exactly one row.

Anchor sketch — four load shapes, four different answers

Legend: boxed = a named node | **reversed = focus node** | **Q = self-test cue**



Q Why is the branch the one that does damage fastest, even though it involves no attacker and no bug?

Q Which branch cannot be caught by testing at all, and what's the reason?

Q Two of these four branches are things *you* caused. Name them and name the person you should be talking to about each.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“Marketing broke the site.”	“A self-denial attack — unthrottled internal demand, amplified by a deep link that bypassed the CDN so every asset came from origin.”
“Everything restarted at once and the database fell over.”	“A dogpile: synchronised transient load with cold caches. Fix with randomised clock slew and jittered exponential backoff.”
“It works fine in the test environment.”	“Test is one-to-one; production is ten-to-one. Unbalanced capacities are a scaling effect — this class of defect can't be tested out, it has to be designed out.”
“The query came back with way too much data.”	“Unbounded result set. It's a handshaking failure — the caller never stated how much it was prepared to accept.”
“The autoscaler went nuts.”	“A control-plane force multiplier: the automation's belief about desired state diverged from reality. It needs a governor with hysteresis and a confirmation step on large gaps.”
“All the servers talk to each other.”	“Point-to-point intra-service communication — connections grow as the square of instance count. Past tens of nodes, move to multicast or publish/subscribe.”
“Our best customers get the worst performance.”	“Expensive users, plus an unbounded traversal — the customers with the most history load the largest result sets. Power-law distribution, not a bell curve.”

> *“Good marketing can kill you at any time.”*

PART 3 · QUESTION 1

Your team ships pagination by storing each user's full search-result set in the session. Load tests at 500 concurrent users pass comfortably. Two months after launch the fleet starts running out of memory every afternoon, and the crashes across instances get closer together each time.

(a) Name the three antipatterns chained together here and the order they fire in. **(b)** Give the cheapest code change that stops today's bleeding. **(c)** Give the change that eliminates the whole class of problem, and say what it costs you.

PART 3 · QUESTION 2

An ops team sets the config-management agent to pull on the hour and the cache-warming job to run at 00:00. Separately, they add an autoscaler that reacts to one-second CPU samples, while a new instance takes five minutes to become useful.

(a) Name the two failure modes you now expect. **(b)** Say which one is a control-plane problem and what makes it different in kind from the other. **(c)** Give one specific safeguard for each, and name the pattern each safeguard belongs to.

2×2 — classify a load event before you react to it

COLUMNS: WHAT SHAPE DID THE DEMAND ARRIVE IN? →

	Sustained growth a trend, over weeks or months	**Synchronised spike** everything in the same minute
FROM OUTSIDE	**Users, and what they cost** Session memory, expensive checkout users, socket and port limits, result sets that grow with your own history. *Move:* capacity modelling; keep sessions thin; paginate everything; load-test at 4–10% conversion, not 2%.	**Flash mob / hostile mob** A celebrity link, a link aggregator, or a botnet DDoS. Session management is the soft spot in both cases. *Move:* shed load at the edge, per-host circuit breakers, CDN and firewall blocks, DDoS appliances baselined for a month first.
FROM INSIDE	**Structural mismatch** Scaling effects, $O(n^2)$ point-to-point, shared-resource fan-in, unbalanced front-to-back capacity ratios. *Move:* design it out — it will not show up in a two-server test environment. Bulkheads, handshaking, back pressure, and a scaled-up QA.	**Self-inflicted spike** Marketing email with a deep link · midnight cron · config push · fleet restart with cold caches · an autoscaler acting on stale truth. *Move:* talk to humans *before* the event; clock slew and jitter; pre-scale rather than autoscale; put a governor on every automated action that runs in the unsafe direction.

Read it as: the left column is an engineering problem you can model. The right column is a coordination problem — the top-right with strangers, the bottom-right with your own colleagues and your own robots. The bottom-right is the one you can actually prevent, which is why it's the focus cell.

CARRY-OUT RULE FROM THIS PART

Bound every number a caller doesn't control. Rows returned, sessions created, retries fired, instances started, bytes cached. “The only sensible numbers are *zero*, *one*, and *lots*” — so unless a query returns exactly one row, it can return too many, and you must say how many you'll take.

4

PART 4 OF 4

The counter-moves: twelve stability patterns

Timeouts · Circuit Breaker · Bulkheads · Steady State · Fail Fast · Let It Crash · Handshaking · Test Harnesses · Decoupling Middleware · Shed Load · Back Pressure · Governor

TENTH-GRADER VERSION

- You can't stop things from breaking. You can stop the break from spreading.
- Two moves cover most of it: **stop waiting**, and **stop calling the thing that keeps hurting you**.
- Then build walls, so one flooded room doesn't sink the ship — and learn to say “no” politely when you're full.
- And keep the system boring: nothing piling up, nobody logging in at 3 a.m. to tidy it by hand.

Start with the house, the ship and the restaurant

- **The fuse** is a component *designed to fail first*, so the wiring doesn't. Its two flaws — you run out of them, and a US penny was the same diameter as one — are why we now use resettable **circuit breakers**. Same principle: detect excess, fail first, open the circuit; and once the danger has passed, reset.
- **A ship's bulkheads** divide the hull into watertight compartments. One penetration doesn't sink the ship. That's damage containment, and it costs you usable space.
- **A full restaurant** turns you away at the door. That's kinder than seating you and never bringing food. And a good chef does *mise en place* — every ingredient on the counter before starting — so a missing one costs seconds, not a ruined service.
- **A steam governor** stops the engine spinning fast enough to tear itself apart, even when the boiler could drive it harder.
- Four everyday objects, four of the twelve patterns. The rest are variations on the same two instincts: *bound the wait*, *contain the damage*.

> *“Hope is not a design method.”*

The twelve, with the neighbour each is confused with

PATTERN	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — THE PATTERN IT'S MISTAKEN FOR
1 · Timeouts outbound	Stop waiting once you've decided the answer isn't coming.	Socket connect and read timeouts; a checkout timeout on every resource pool; always the mutex form that takes a timeout argument. Package the whole “check out connection → query → map results → check in” sequence into one gateway so you get the error handling right in exactly one place.	vs **Fail Fast**: Timeouts protect you from *someone else's* failure and apply to **outbound** calls. Fail Fast tells your *callers* quickly and applies to **inbound** requests. “Two sides of the same coin.” **On retries:** immediate retry usually hits the same problem and just makes the caller wait longer. Prefer **queue-and-retry later** — store-and-forward, the way email works.

2 · Circuit Breaker outbound	After enough failures, stop making the call at all for a while.	**Closed** (pass through, count faults) → threshold → **Open** (fail immediately, don't even try) → after a timeout → **Half-Open** (let one trial call through) → success resets to Closed, failure returns to Open. Fallbacks: last good value, a cached value, a generic non-personalised answer, or a secondary provider.	vs **retry**: retries *re-execute* operations; breakers *prevent* them. Count **fault density**, not raw totals — five faults in thirty seconds is a different animal from five spread over five hours. Use a Leaky Bucket: increment on fault, a background timer decrements toward zero. Scope it to **one process**. Sharing state across processes adds an out-of-process call — your safety mechanism gains a new failure mode. Log every state change and expose it; state-change frequency is a leading indicator of trouble elsewhere. Ops must be able to trip and reset it by hand. Deciding what happens when it's open is a **business** decision — take the order without confirming stock? — so bring stakeholders in.
3 · Bulkheads containment	Partition the system so one flooded compartment doesn't sink it.	Granularities, small to large: thread pools inside a process (always reserve one for admin requests, so a fully hung server can still be asked for a thread dump or a shutdown) → CPU core binding → separate instances → dedicated server farms per critical client → AWS zones and regions → one compartment per function invocation in FaaS. Airline example: dedicated check-in servers keep working while shared servers drown in flight-status queries during a storm.	vs **Circuit Breaker**: a bulkhead is **provider-side partitioning**; a breaker is **caller-side abstention**. A bulkhead won't help the callers of whichever partition *does* go down — that's what the breaker is for. Note that redundant VMs are weaker than redundant physical machines: most provisioning tools won't guarantee physical isolation. With software-defined load balancers, bulkheads can be **dynamic** — carved out and destroyed as traffic patterns change.
4 · Steady State hygiene	For every mechanism that accumulates a resource, something must recycle it.	Three kinds of sludge. **Data:** purge with *application* logic — the DBA's script doesn't know how the app behaves when rows vanish, and an ORM handles dangling references far worse than a document store. **Log files:** rotate by size; on UNIX the last 5–10% is reserved for root, so I/O errors start at 90–95% full; ship logs off-host to a central store; keep compliance retention on non-production machines. **Caches:** bound the size and invalidate.	The target is "**no fiddling**": run at least one full deployment cycle with nobody logging in. Every human touch is an opportunity for the "ohnosecond" — one engineer resilvered a root mirror the wrong way round and instantly annihilated a live OS. Servers that need hand-holding become *pets*, and admins stay logged in permanently. Immutable infrastructure is the logical endpoint: it can't be fiddled with. Cache questions to ask: **is the key space finite?** and **do the items change?** If the keys are unbounded you need a size limit and an invalidation strategy — nine times out of ten a periodic flush beats LRU. Don't cache things that are cheap to generate (one system cached hundreds of single space characters), monitor hit rates, and build caches on weak references so the collector can help rather than being obstructed.

5 · Fail Fast inbound	If you can tell in advance you'll fail, say so immediately instead of failing slowly.	A load balancer with no healthy servers should **refuse** the connection, not queue it hopefully. A service can check out the connections it will need and check the state of its circuit breakers up front — *mise en place*. Validate parameters in the controller before touching the database. The photo-rendering case: pre-check every font, image, background and alpha mask and preallocate memory, and the software-induced reprint rate went to zero. The one resource they *didn't* preallocate — disk space — was the one place they still failed late, minutes into a render.	vs **Timeouts**: inbound versus outbound (see pattern 1). **The nuance that matters:** report a *system* failure (resources unavailable) differently from an *application* failure (bad parameters). A generic “error” lets a user mashing Reload with invalid input trip an upstream circuit breaker.
6 · Let It Crash recovery	Accept component-level instability to get system-level stability: the cleanest state a program ever has is right after startup.	Four things must be true. **Limited granularity** — a boundary for the crashiness (an actor in Erlang/Elixir; an actor library like Akka elsewhere, with extra code review because the runtime won't enforce the rules). **Fast replacement** — Go in a container restarts in milliseconds, so crash the container; NodeJS on a long-lived VM restarts in milliseconds but the VM takes minutes, so crash the process only; a JavaEE app with minutes of startup means this pattern is simply wrong for you. **Supervision** — a hierarchical supervisor tree that can restart a child, restart all children, or crash itself upward. **Reintegration** — the restarted thing must rejoin: circuit breakers reintegrate direct callers automatically, load-balancer health checks re-add pool members.	vs **Fail Fast**: Fail Fast refuses *work*; Let It Crash discards *state*. vs an **autoscaler or PaaS restart**: those look like supervision but **lack discretion** — they'll restart a crashing instance forever, and there's no hierarchy. A real supervisor tracks restart *density* and crashes itself if restarts get too dense, because that means either the state isn't really being cleaned or the whole system is in trouble and the supervisor is masking it. And: “the supervisor is not the service consumer. Managing the worker is different than requesting work.”
7 · Handshaking cooperative	Let the receiver tell the sender whether it's ready.	Universal below the application layer — RS-232 flow control, the TCP three-way handshake, the TCP receive window telling a sender to stop. Nearly absent above it. The practical form you can actually build today: a **health-check endpoint** the load balancer polls, returning 503 when the instance is busy so the balancer stops sending it work.	vs **Circuit Breaker**: handshaking is **cooperative** — ask before you call. A breaker is the **unilateral stopgap** for providers who can't handshake: you just make the call and track whether it worked. Why it's rare: HTTP defines 503 for exactly this, but most clients treat anything that isn't 200, 403 or 302 as fatal — some treat other 2xx codes as errors. So build handshaking into any low-level protocol you design yourself.

8 · Test Harnesses verification	A deliberately malicious fake for the far end of every integration point.	One “killer” server, a different misbehaviour per port: 10200 accepts and never replies; 10201 replies with bytes from `/dev/random`; 10202 opens the connection then drops it. Failures worth emulating: refuse the connection; leave it in the listen queue; SYN/ACK then silence; nothing but resets; advertise a full receive window and never drain; connect but send no bytes; lose packets to force retransmit delays; send headers and no body; send one byte every thirty seconds; send HTML where XML was promised; send megabytes where kilobytes were expected; refuse all credentials. Have the harness log requests — your application may die without a whimper.	vs **mock objects**: a mock can only misbehave *within the defined interface*. A harness is a separate server, so it owes no allegiance to any interface and can violate the network, protocol and application layers alike. vs an **integration-test environment**: that can only show you failures the remote system is *specified* to produce — layer seven only, and not even all of it. It also forces version-locking across the whole company. It **supplements**, never replaces, unit, acceptance and penetration tests. Follow this road far enough and you arrive at chaos engineering.
9 · Decoupling Middleware architecture	Choose how tightly caller and receiver are bound — in time as well as in space.	The spectrum, tight to loose: in-process calls → interprocess (pipes, shared memory, semaphores) → remote procedure calls (DCE RPC, DCOM, RMI, XML-RPC, HTTP — **yes, JSON over HTTP too**) → message-oriented middleware (MQ, publish/subscribe, SMTP, SMS) → tuple spaces (JavaSpaces, GigaSpaces).	Synchronous request/reply means both systems must be alive *at the same moment*. “Synchronous calls are particularly vicious amplifiers that facilitate cascading failures.” Message-oriented middleware decouples in space **and time** and therefore **cannot produce a cascading failure**. The price is real: asynchronous design means exception queues, late responses, callbacks both machine-to-machine and human-to-human, and business decisions about acceptable financial risk. Every other pattern here can be added without redesigning the system; **this one ripples through everything**, so it's a near-irreversible decision to make early rather than late.
10 · Shed Load inbound, at the edge	Refuse new work when load is too high — copy what TCP does with the listen queue.	Ideal trigger: your own response time measured against your SLA. Simpler: a semaphore capping concurrent in-flight requests. With a load balancer in front, return 503 on the health-check page to buy breathing room; balancers are very good at recycling connections quickly.	vs **Back Pressure**: **shed load at the edges**, where demand is uncontrolled and the consumer pool is effectively infinite. **Back pressure inside a system boundary**, where consumers are finite and known. Why it's needed at all: services fall over *before* the connection queue fills, because of contention for pooled resources. “To an outside observer, there's no difference between ‘really, really slow' and ‘down'.” And: **“You can't out-scale the world.”**

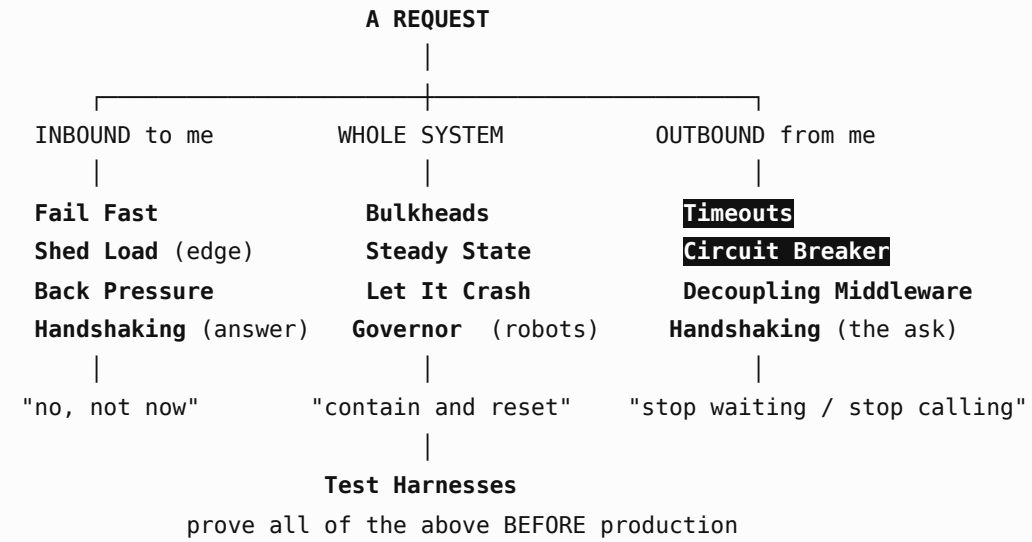
11 · Back Pressure inbound, inside	Block the producer until the consumer catches up.	Every performance problem starts with a queue backing up somewhere. An unbounded queue consumes all memory, and by Little's law an unbounded queue means unbounded response time. So bound it — and then choose, because when a bounded queue is full you have exactly four options: pretend to accept and silently drop; accept and drop something already queued; refuse; or **block the producer**. That fourth one is back pressure. TCP does it with the window field. Example: allow each API server 100 concurrent calls to the storage engine; the 101st caller blocks. Better still, move the blocking to the receiver so it can tune its own queue depth for maximum throughput rather than every server guessing.	Back pressure **produces blocked threads on purpose** — so distinguish a temporary condition from a consumer that is simply broken. At a system edge, blocked threads frustrate users or provoke retry loops, so accept requests on one thread pool and make outbound calls on another: then the request thread can time out and return **503**, or enqueue the work and return **202**. Wire it to monitoring. Back pressure engaging is real information — either a trend or a consumer that went rogue.
12 · Governor control plane	A stateful, time-aware, **asymmetric** limiter on automated actions, whose purpose is to buy humans time.	Reddit's fix after the outage: the autoscaler may only shut down a certain percentage of instances at a time. Unsafe directions to slow down: shutting instances off, deleting data, blocking client IPs.	vs a **rate limiter**: a plain rate limiter is symmetric. A governor is deliberately asymmetric — fast in the safe direction, slow in the unsafe one. vs a **circuit breaker**: a breaker stops *calls* after observed failures. A governor slows *your own privileged actions* before any failure has occurred. Safety is **U-shaped**, not one-directional: shutting instances down is unsafe for availability, spinning them up is unsafe for cost. So actions are safe within a range and meet increasing resistance outside it — that's the governor's response curve. And it's pointless unless it's wired to alerting: the entire purpose is to get a human involved while intervention is still possible rather than recovery.

Trade-offs — what each pattern costs you

PATTERN	BUYS YOU	COSTS YOU	WRONG CHOICE WHEN…
Timeouts	Fault isolation — someone else's problem stops being yours.	Half your code can end up being error handling. Some slow-but-valid work gets cut off.	Never wrong. But a value plucked from the air rather than derived from your SLA is close to useless.
Circuit Breaker	Stops you hammering a sick provider; degrades function automatically.	The fallback is a business decision, not a technical one. Per-process state means every instance rediscovers the outage independently.	The provider can genuinely handshake — then cooperate instead of guessing.
Bulkheads	Partial service survives. Critical callers keep their capacity.	Reserved capacity sits idle; more partitions to operate and monitor.	Capacity is so tight that reserving any of it guarantees the outage you're trying to avoid.
Steady State	No 3 a.m. chores; no admins permanently logged into production.	Purge logic is nasty, detail-oriented work that needs real coding and testing — which is exactly why it always slips past launch.	Never. The “we'll add purging six months after launch” plan has never once survived contact with the first emergency release.
Fail Fast	No wasted work; callers get their capacity back immediately.	You must separate system errors from application errors, or you'll trip upstream breakers on ordinary bad input.	The pre-check costs more than the work itself.
Let It Crash	The cleanest possible state, reached fast, without unreliable error recovery.	Requires all four preconditions. Without supervision it's a fork bomb; without reintegration it's a shrinking cluster.	Monoliths, heavy runtimes, anything with startup measured in minutes. “Don't crash monoliths.”
Handshaking	Cooperative demand control — the politest form of protection.	Both ends must implement it, and common protocols barely support it.	Third-party providers you don't control. Use a Circuit Breaker instead and stop asking.
Test Harnesses	The only practical way to meet out-of-spec failures before production does.	A second codebase, deliberately hostile, that someone must maintain.	As a *replacement* for other testing. It supplements.
Decoupling Middleware	Removes whole classes of failure — integration points, cascading failures, slow responses, blocked threads — at once.	The hardest change here, and close to irreversible. Asynchronous design is genuinely harder to reason about.	Logical simplicity of request/reply matters more than the coupling — and you've said so out loud, knowingly.
Shed Load	Response times stay sane no matter what the world does.	Some real users get refused.	Inside a boundary with finite consumers — back pressure is more efficient there.
Back Pressure	Balanced throughput across synchronously-coupled services.	Deliberate blocked threads; frustrated users if it reaches an edge.	Across a system boundary, or when your user base is the open internet.
Governor	Humans get time to look at the whole situation before automation finishes the job.	It slows legitimate emergency automation too.	Applied to the *safe* direction. Governors are asymmetric on purpose.

Anchor sketch — where each pattern sits relative to the call

Legend: boxed = a named node | **reversed = focus node** | **Q = self-test cue**



Q Handshaking appears in two columns. Why — and which side implements which half?

Q Which of the twelve is an *architecture* decision rather than an implementation one, and what follows from that?

Q Name the two that are useless without a fourth precondition each, and name those preconditions.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“We stopped calling it after it kept failing.”	“We wrapped it in a circuit breaker. It trips on fault density rather than raw count — leaky bucket — and every state change is logged and exposed to monitoring.”
“We split it up so one part dying doesn't kill everything.”	“We introduced bulkheads — at the thread-pool granularity inside the process, and at the availability-zone granularity outside it.”
“Tell them no instead of making them wait.”	“Fail fast on inbound requests, shed load at the edge, apply back pressure inside the boundary.”
“Just restart it.”	“Let it crash — but only with limited granularity, fast replacement, real supervision, and automatic reintegration. Otherwise it's a fork bomb.”
“The queue got huge and everything slowed down.”	“Unbounded queue, so by Little's law response time is unbounded too. Bound it and pick a full-queue policy: drop new, drop old, refuse, or block the producer.”
“We put a limit on the autoscaler.”	“We added a governor — stateful, time-aware, asymmetric, with a U-shaped response curve and an alert when it engages.”
“We tested it against a fake that misbehaves.”	“We built a test harness — a separate server, so it can violate the protocol, not just the interface, the way a mock is limited to.”
“We should use queues instead of HTTP calls.”	“Decoupling middleware — moving from same-time, same-process coupling toward message-oriented middleware, which decouples in space and time and therefore can't cascade. It's an architecture decision, so it's near-irreversible.”

> *“Judiciously applying these stability patterns results in software that stays up, come hell or high water.* ***The key to applying these patterns successfully is judgment.*** *Examine the software's requirements cynically. View other enterprise systems with suspicion and distrust — any of them can stab you in the back.* ***Paranoia is good engineering.****”*

PART 4 · QUESTION 1

A checkout service calls three things: **(i)** an inventory service your own team owns, **(ii)** a third-party tax API with a hard rate limit and no health endpoint, **(iii)** an internal fraud model that is slow but usually correct. Overnight, traffic doubles because of a marketing email nobody warned you about.

(a) Choose one primary pattern for each of the three dependencies and justify each in one line. **(b)** Name the pattern you'd apply to your own *inbound* traffic, and say why it is not the same one you used for (ii) even though both are about “too much load”. **(c)** Name the conversation you should have had last week and with whom.

PART 4 · QUESTION 2

A colleague proposes: “Let's wrap every outbound call in a circuit breaker, and store the breaker state in a shared Redis so all instances agree on when it's open. And let's trip it at ten failures total.”

(a) Give the two objections — one about *where the state lives*, one about *how you count*. **(b)** Name the mechanism you'd propose for counting instead, and describe it in one sentence. **(c)** The team's real requirement is “we want to see globally when a provider is sick.” Satisfy that requirement *without* sharing breaker state.

2×2 — pick the right pattern in one step

COLUMNS: WHAT ARE YOU TRYING TO DO ABOUT THE LOAD? →

	Refuse fast give an answer now, even if it's “no”	**Absorb & isolate** keep working at reduced function
CALLER side outbound	**Timeouts + Circuit Breaker** Bound the wait; then stop making the call at all once faults are dense. Together they are the default answer for every integration point. *Also:* Handshaking, when the provider can tell you it's not ready.	**Decoupling Middleware** Don't wait at all — move to messaging, so caller and provider needn't be alive at the same moment. Plus **delayed, queued retries** rather than immediate ones. *Cost:* an architecture decision, not a code change.
PROVIDER side inbound	**Fail Fast · Shed Load** Check resources up front (*mise en place*) and refuse before you burn a thread. Shed load at the edge where demand is uncontrolled; 503 on the health check tells the balancer to back off. *Watch:* report system errors differently from user errors.	**Bulkheads · Back Pressure · Let It Crash · Steady State · Governor** Partition capacity so a flood is contained; slow producers down inside your boundary; discard bad state fast and reintegrate; keep the sludge from accumulating; put a governor on anything automated that acts in an unsafe direction.

Read it as: the left column returns an answer; the right column changes the structure. **Test Harnesses** sits outside the grid because it's how you find out whether any of the other eleven actually work.

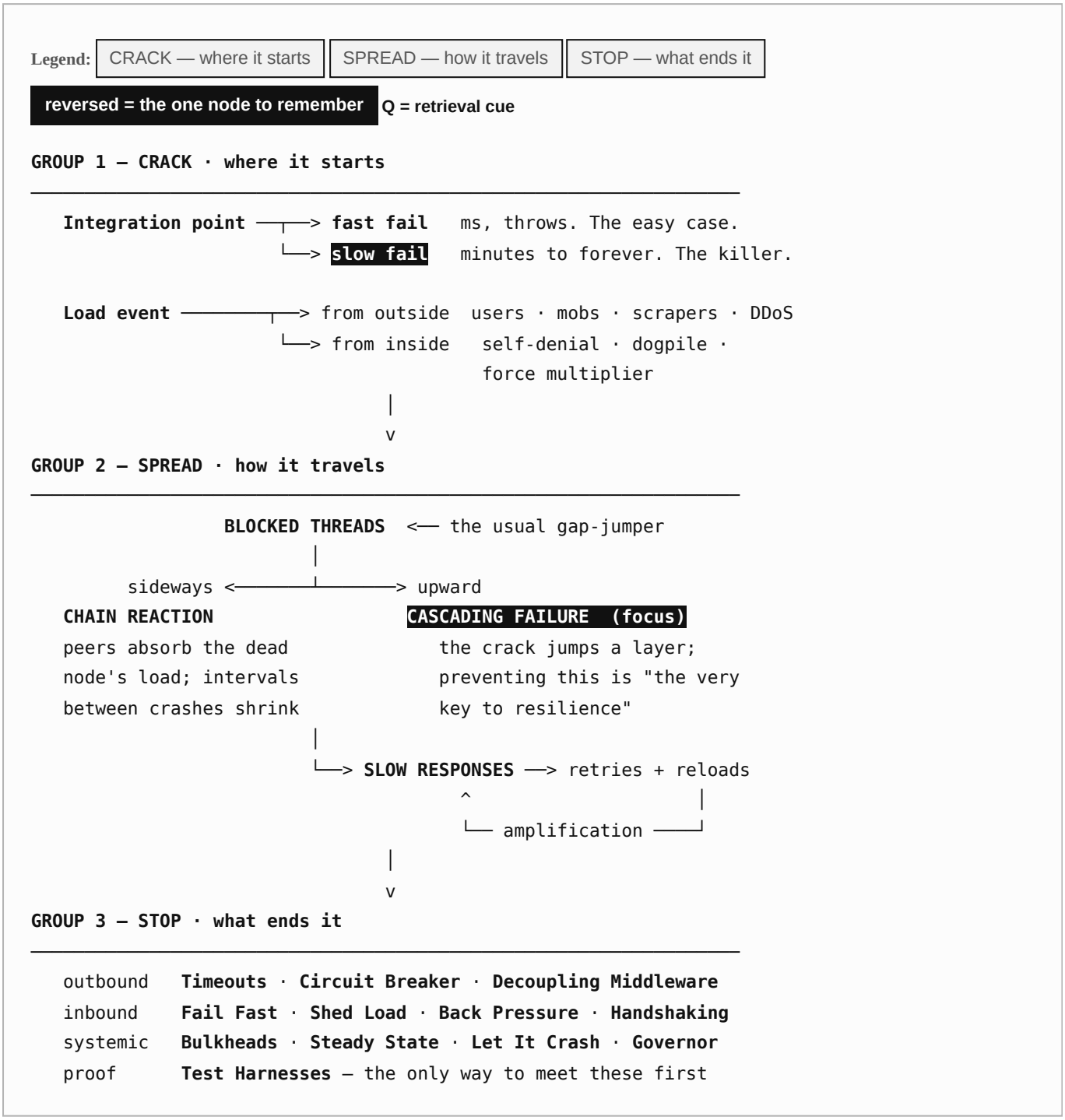
CARRY-OUT RULE FROM THIS PART

Two patterns, then the rest. If you can only do two things to a system, do Timeouts and Circuit Breaker on every outbound call — they cover integration points, cascading failures, blocked threads and slow responses at once. Everything else is refinement. And resist counting: “count of patterns applied” is never a quality metric.

SECTION 6

Concept sketch — the whole competency on one page

Three named groups, in the order the story actually runs: **CRACK → SPREAD → STOP**. Every node carries a question. Cover the answers, walk the map, and each answer hands you the next question.



The question per node — cover the right column

NODE	QUESTION	ANSWER IN ONE LINE
Integration point	Why is it called the number-one killer?	Every socket, pipe, process and remote call can and will hang, and a stand-alone system is almost useless — so you can't avoid them.

Fast fail vs slow fail	Which one do you want, and why is that counter-intuitive?	Fast. It throws in milliseconds and returns the thread. Slow failure holds capacity for minutes or forever — “a slow response is a lot worse than no response.”
Load from outside	What's the one architectural component all four external threats attack?	Session management — sessions are the Achilles' heel of a server-side web application.
Load from inside	What makes self-inflicted load the most preventable kind?	Somebody in your building already knows it's coming. Communication is the fix; pre-scaling is the backup.
Blocked Threads	Why is this called the proximate cause of most failures?	Because “gradual slowdown” and “hung server” both reduce to it, and it's the mechanism that lets a crack jump a layer boundary.
Chain Reaction	Direction, cause, and the only real cure?	Sideways among peers; the dead node's load lands on identical code with the same defect; only fixing the defect cures it — bulkheads just slow it.
Cascading Failure	Why is this *the* node to remember?	“Preventing cascading failures is the very key to resilience.” It's the number-one crack accelerator, and Timeouts + Circuit Breaker are its direct answer.
Slow Responses	Why do slow responses *increase* load?	Callers retry, users hit Reload, speculative retries multiply — the response to slowness generates more demand.
Timeouts + Circuit Breaker	What's the division of labour between them?	Timeouts tell you there's a problem; the breaker decides to stop calling. Timeouts bound each call; the breaker bounds the *number* of calls.
Fail Fast vs Timeouts	The one-line distinction?	Timeouts for outbound calls (protect me from your failure); Fail Fast for inbound requests (tell you quickly I can't). Two sides of the same coin.
Shed Load vs Back Pressure	Where does each belong?	Shed load at the edge, where the consumer pool is the internet. Back pressure inside a boundary, where consumers are finite and can actually be slowed.
Bulkheads	What does it protect, and what does it explicitly *not* protect?	It preserves partial function inside your system. It does nothing for the callers of the partition that died — that's the Circuit Breaker's job.
Governor	What makes it different from a rate limiter?	It's stateful, time-aware and asymmetric — fast in the safe direction, increasingly resistant in the unsafe one, and wired to an alert.
Test Harnesses	Why can't an integration environment do this job?	An integration environment can only produce failures the remote system is *specified* to produce — application layer only. A harness is a separate server and owes no interface any loyalty.

Master 2×2 — the decision card you can carry to other problems

This one isn't about software specifically, which is the point. Two structural properties decide whether small faults stay small — in a payments platform, a hospital ward, a supply chain, or a power grid.

COLUMNS: INTERACTIVE COMPLEXITY — CAN ANYONE HOLD AN ACCURATE MODEL OF THE SYSTEM? →

	LOW complexity effects are visible and traceable	**HIGH complexity** hidden linkages; operators' instincts are wrong
LOOSE coupling	**Boring. Aim here.** Failures stay local and legible. You lose a component, not a system. *Keep it here by:* messaging over synchronous calls, shared-nothing where you can afford it, bulkheads at natural boundaries.	**Messy but survivable** You won't understand it, but slack in the joints absorbs the surprise. *Invest in:* observability and external monitoring — you can't reason your way to the fault, so you must be able to see it.
TIGHT coupling	**Fast and brittle** Predictable, so you can enumerate the failure modes — but they arrive at full speed. *Invest in:* timeouts, fail fast, bounded queues. Cheap, and enough at this altitude.	**The technology frontier** "High interactive complexity and tight coupling conspire to turn rapidly moving cracks into full-blown failures." Three Mile Island lives here. So does most of modern distributed computing. *Nothing but structure will save you:* circuit breakers, bulkheads, governors, back pressure, and rehearsed recovery. Fixing individual bugs will not move you out of this box.

The move that matters: you rarely reduce complexity — systems only grow. You *can* almost always reduce coupling, which means the realistic escape from the bottom-right is upward, not leftward. That is exactly what asynchronous messaging, bulkheads, and circuit breakers do: they buy looseness.

THE THUMB RULE, IN ONE SENTENCE

Bound every wait, every queue, every result set, and every automated action — then partition whatever is left.

Applies far outside software. A hospital triage desk bounds the wait. A factory's kanban bounds the queue. A bank's daily transfer cap bounds the automated action. A ship's bulkheads partition what's left. Every one of them is the same idea: *put a limit on the thing that would otherwise grow until it kills you.*

> *"In time, even shockingly unlikely combinations of circumstances will eventually occur… a single small service might do ten million requests per day over three years — more than ten billion opportunities for bad things to happen. Astronomical observations indicate there are four hundred billion stars in the Milky Way.* ***Astronomically unlikely coincidences happen all the time.****"*

Symptom → diagnosis → pattern

The on-call page. Cover the middle and right columns and work down the left.

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO APPLY
Process up, port open, logs quiet, no CPU — and no responses.	**Blocked Threads.** Everything is waiting on something that will never come.	Timeouts on every wait; external synthetic monitoring so you find out from outside; proven concurrency primitives.
One node dies, then the rest, with shrinking gaps between deaths.	**Chain Reaction** from a load-related defect — usually a memory leak or a race condition.	Fix the defect. Bulkheads to split the layer. Health-checked autoscaling, if the scaler beats the reaction.
A dependency died and your service died with it, though your own resources looked fine.	**Cascading Failure** through a drained resource pool.	Circuit Breaker plus Timeouts. Cap how long a thread may wait to check out a resource.
Latency climbs steadily, then callers start timing out, then their callers do.	**Slow Responses** propagating upward — often GC from a leak, sometimes a TCP stall from a hand-rolled protocol.	Fail Fast against your own SLA; hunt the leak; Shed Load; check that reads loop until the receive buffer is drained.
Every request-handling thread is in the same third-party library.	**Integration Point** plus a library that pools internally and won't let you configure failure modes.	Timeouts if it exposes them; otherwise wrap it with your own worker pool and a future — and push the vendor.
Hangs at the same time every day, cleared by a restart.	Idle connections reaped by a firewall or NAT device — **the 5 a.m. problem**.	Keep-alive / dead-connection detection to refresh the middlebox timer; socket timeouts; a packet capture to confirm.
Memory climbs all afternoon; OOM; restart; repeat tomorrow.	**Users** (sessions) or an unbounded cache. Possibly an **Unbounded Result Set** feeding it.	Keep sessions thin; bound cache size and invalidate; paginate; weak references as a last resort; Steady State.
The database is fine, but one query returns millions of rows.	**Unbounded Result Set** — a handshaking failure in disguise.	`LIMIT` / `TOP` / `rownum`; pagination in the API contract; production-sized test data.
Traffic spike exactly on the hour, or right after a deploy.	**Dogpile** — synchronised transient load, usually with cold caches.	Randomised clock slew; jittered exponential backoff; warm caches before accepting traffic; random deltas in load tests.
Traffic spike right after a company announcement.	**Self-Denial Attack.**	Talk to marketing. No deep links in mass mail; send in waves; static landing zones; pre-scale in advance.
Your automation did something enormous and instantaneous.	**Force Multiplier** — the control plane's belief diverged from reality.	Governor with hysteresis; confirmation on large expected-vs-observed gaps; distrust any observation claiming >80% of the fleet is down.
Back end falls over whenever the front end has a good day.	**Unbalanced Capacities** — a scaling effect that QA's 1:1 ratio hides.	Circuit Breaker on the caller; Handshaking and Back Pressure on the provider; Bulkheads to reserve capacity for critical callers.
It only breaks in production, at production size.	**Scaling Effects** — a many-to-one relationship that grew.	Design it out; you cannot test it out. Replace point-to-point with multicast or pub/sub; make shared resources redundant and non-exclusive.
Somebody has to log in every night to clean something up.	**Steady State** violation — sludge accumulating faster than it's recycled.	Application-level purge logic; log rotation and off-host shipping; bounded caches with an invalidation strategy.

Restarts help for a while, and nobody knows why.	A resource leak. The restarts are papering over a Steady State violation and probably a coming Chain Reaction.	Find the leak; add the recycling mechanism; only then consider Let It Crash — and only with supervision and fast replacement.

THE MAPPING IN THE OTHER DIRECTION — WHICH ANTIPATTERNS EACH PATTERN ACTUALLY COUNTERS

PATTERN	COUNTERS
Timeouts	Integration Points · Blocked Threads · Slow Responses · (thereby) Cascading Failures
Circuit Breaker	Integration Points · Cascading Failures · Unbalanced Capacities · Slow Responses · abusive clients
Bulkheads	Chain Reactions · Unbalanced Capacities · shared-service blast radius
Steady State	Slow Responses · memory leaks · the “daily restart” habit
Fail Fast	Slow Responses · Cascading Failures · capacity loss under partial failure
Let It Crash	Blocked Threads · corrupt state · unreliable error recovery
Handshaking	Unbalanced Capacities · Slow Responses · Cascading Failures
Test Harnesses	Integration Points — by finding them before production does
Decoupling Middleware	Integration Points · Cascading Failures · Slow Responses · Blocked Threads
Shed Load	Users · Slow Responses · traffic beyond any possible capacity
Back Pressure	Unbalanced Capacities · unbounded queues · Slow Responses
Governor	Force Multiplier · runaway automation · runaway cost

SECTION 8

Numbers worth remembering

Not trivia — each one is the number that makes an argument land in a design review.

NUMBER	WHAT IT'S ABOUT	WHY IT MATTERS IN AN ARGUMENT
12.5% → 14.3%	Per-node load in an 8-server farm before and after one death.	Only 1.8 points of total load, but a **15% jump** on each survivor. That's how a chain reaction starts from a "small" failure.
50% → 100%	Two-node cluster losing one node.	The degenerate case: the survivor's workload **doubles**. Two-node redundancy is barely redundancy.
3,000 vs 450	Front-end threads vs back-end threads in the unbalanced-capacity example (20 hosts/75 instances vs 6 hosts/6 instances).	"Three thousand threads calling into seventy-five threads is not in the ballpark." Capacity modelling has to be a ratio, not two separate numbers.
~10 ms	TCP three-way handshake on the same switch.	The baseline that makes "connection refused" a cheap failure — and makes a multi-minute connect obviously pathological.
20 / 30 minutes	Linux 2.6 with `tcp_retries2=15`; HP-UX of the era.	How long a single socket write can block when packets are silently dropped. Reads can block **forever**.
64,511	Max connections on one IP using ports 1024–65535 (IANA ephemeral range is 49152–65535 ≈ 16,383).	Why a million-connection server needs about **16 virtual IPs** and serious kernel buffer tuning.
$O(n^2)$	Connection growth for point-to-point communication inside a service.	Fine at two nodes, painful at a hundred. The number that ends the "we'll just have them talk to each other" conversation.
2% → test at 4–10%	Typical retail conversion rate vs what you should load-test.	Buyers are the expensive users. Test the expensive path at several times its expected share — but don't *capacity-plan* at 100% or you'll overspend tenfold.
90–95%	Filesystem fullness at which a UNIX application starts getting I/O errors (the last 5–10% is reserved for root).	Log rotation isn't housekeeping; it's the thing standing between you and a reentrant exception handler consuming eight CPUs.
10–100 ms	"Fast enough" for a service behind a web API.	Beyond that you start losing capacity and customers. It's also the sane basis for a Fail Fast threshold.
>80% unavailable	The control-plane heuristic.	If your monitoring says more than 80% of the system is down, suspect the **observer**. This one line would have prevented the service-discovery partition failure.
768,000	Angry Facebook users per day at Six Sigma quality (200 requests/page × 1.13bn daily users × 3.4 defects per million).	Why "five nines" stopped being an impressive target at scale.
10.95 billion	Requests in three years for a service doing 10M/day.	"Astronomically unlikely coincidences happen all the time." The rebuttal to "that can't happen."

Glossary — the terms you should be able to define cold

TERM	DEFINITION, AND THE LINE THAT SEPARATES IT FROM ITS NEIGHBOUR
Fault / crack / failure	A fault is the underlying defect or event. A crack is the fault beginning to have effects. A failure is the crack having propagated far enough to stop transactions. The whole book is about the middle step.
Tight coupling	A failure in one component forces load, delay or change onto others. Not the same as *dependency* — you can depend on something loosely, through a queue.

Interactive complexity	Enough moving parts and hidden internal dependencies that operators' mental models are incomplete or wrong, so their instincts do harm. This is what turns a minor fault into “problem inflation”.
Horizontal / vertical scaling	Horizontal adds servers (farms); vertical adds cores, memory and storage to one host. Horizontal gives fault tolerance through redundancy — and gives you chain reactions.
Fan-in / fan-out	Fan-in is how many call *you*; fan-out is how many *you* call. High fan-in means wide blast radius. Reducing fan-in on a shared resource is the practical approximation of shared-nothing.
Butterfly / spiderweb	Two shapes of integration diagram: one central system with fan-in on one side and fan-out on the other, versus many boxes with many dependencies. The constant across both: **connections outnumber services** — roughly 2N for a butterfly, up to N^2 for a web.
Listen queue	The kernel's per-port waiting room for half-open connections. Full = fast refusal (good). Occupied = blocked in `open()` for minutes (bad).
Bogon	A stray packet that arrives late and out of sequence, possibly after the connection closed. The reason TIME_WAIT exists. Real on the open internet, largely academic inside a data centre.
Shared-nothing	Each server runs without knowing about any other. The hypothetical ideal for horizontal scaling; you pay for it in failover.
Control plane	Software that manages infrastructure rather than serving users: logging, monitoring, schedulers, scalers, load balancers, configuration management. It senses state, compares to desired state, and acts — which is why it can act catastrophically.
Hysteresis	Deliberate asymmetry in a control system's response: quick in the safe direction, slow in the unsafe one. Start machines fast, stop them slowly.
Leaky Bucket	A counter incremented on each fault and decremented by a background timer. Measures fault *density* rather than a running total — the right input for a circuit breaker.
Little's law	Queue length, arrival rate and time-in-system are locked together. Practical consequence: an unbounded queue means an unbounded response time. Queues must be finite for latency to be finite.
Liskov substitution principle	Any property true of type T must hold for its subtypes. A subclass that adds `synchronized` breaks it — “functional behavior composes, but concurrency does not compose.”
CQRS	Command Query Responsibility Separation: immutable objects for querying and rendering; state changes issued as command objects. Avoids synchronising domain objects, which breaks across servers anyway.
Mise en place	The chef's discipline of assembling every ingredient before cooking. The Fail Fast pattern in three words.
Pets vs cattle	Servers you nurse individually versus servers you replace. Pets invite fiddling; fiddling invites the “ohnosecond”.
Dogpile	Synchronised transient load — a fleet booting, a cron hour, a config push, a logjam breaking. Costs you peak capacity you'd never otherwise need.
Fight Club bug	Increased front-end load causing exponentially increasing back-end work, usually via a shared resource touched per request.
Power law vs bell curve	Relationship counts in social systems follow a power law, so an entity with a million times the average *will* exist. Test data built on a bell curve will never show you this.

Answer key

PART 1 · Q1 — THE FRAUD-SCORING API

(a) A **slow failure at an integration point** producing **blocked threads**. The provider accepted the connection and never responded, or the connection was silently torn down by something in the middle; either way the calling threads are parked inside a socket read.

(b) The absences are the evidence. **No errors** means nothing has thrown yet, which rules out fast failure — a refused connection or a reset would have raised an exception immediately. **No CPU and no memory movement** rules out a leak, garbage-collection thrash, or an infinite loop; the threads aren't working, they're waiting. A process monitor will call this instance healthy, so you need thread dumps or an external synthetic transaction to see the truth. This is the "dog that didn't bark" pattern from the 5 a.m. story.

(c) The **socket read timeout** and the **connect timeout** on the HTTP client — and behind them, the **checkout timeout** on any connection pool. Bet on the read timeout being unset or infinite: that's the common default, and it's the one that produces exactly this symptom. Full marks also for noting that if the client library hides the socket you may not be able to set it, which is itself the finding.

PART 1 · Q2 — THE MONDAY-MORNING HANG

(a) The connection sat idle over the weekend and a **firewall or NAT device reaped it from its state table**, while both endpoints still believe the connection is valid. This is the 5 a.m. problem with a weekly period instead of a daily one.

(b) Take a packet capture on both sides. The confirming observation is **your side retransmitting the same segment with no response and no ICMP unreachable**, and simultaneously **nothing at all arriving at the partner's interface**. The silence on the far side is the proof: the packets are being dropped in the middle, not rejected at the end. (This also explains why the partner's logs are honestly clean — their application never saw a thing.)

(c) Cheapest fix: a **keep-alive / periodic ping** on the idle connection, at an interval shorter than the middlebox's idle timeout — the mechanism Oracle's dead connection detection provided. Configuring a read timeout is a necessary companion, so the failure at least becomes fast.
Why the obvious fixes fail: a **validation query before checkout** travels over the same dead connection, so it hangs exactly as the real query would. **Closing connections older than an hour** requires sending a teardown packet over the same dead connection — which also hangs. Both "fixes" move the hang, they don't remove it.

PART 2 · Q1 — TWELVE INSTANCES IN SEVEN MINUTES

(a) A **Chain Reaction** in the recommendations layer and a **Cascading Failure** into the product-page service.

(b) The chain reaction moved **sideways**, carried by **redistributed load**: each death handed its share to identical peers running the same defective code, so every survivor got closer to the same cliff. The shrinking gaps between failures are the signature — that acceleration is exactly what the search-farm story charted. The cascade moved **upward**, carried by **blocked threads**: the product-page service's threads were parked waiting on a service that had stopped answering, which is why its CPU and memory looked fine while its thread count pinned.

(c) Against the chain reaction: **fix the underlying load-related defect** — a leak or a race. Structurally, **Bulkheads** split the layer into pools so one chain reaction becomes two slower ones; health-checked **autoscaling** helps if the scaler reacts faster than the reaction propagates. Against the cascade: **Circuit Breaker plus Timeouts** on the calling side.
Why swapping them fails: a circuit breaker in the recommendations layer wouldn't have saved *it* — the peers weren't calling each other, they were absorbing traffic. And bulkheads in the recommendations layer do nothing for its callers; bulkheads explicitly won't help the callers of whichever partition does go down — Circuit Breaker on the calling side is what covers them.

PART 2 · Q2 — THE SYNCHRONIZED CACHE

(a) One thread entered the inherited `synchronized get()`, called the dead partner API, and blocked there indefinitely while holding the monitor — so every other thread that touched the cache queued behind it, and eventually every request-handling thread on the box was blocked, regardless of which feature it was serving.

(b) The **Liskov substitution principle**. Any property true of a type should hold for its subtypes, but the subclass turned a fast in-memory lookup into a blocking remote call under the same lock. The compiler allows it because Java and C# don't treat synchronisation as part of a method's contract — you can't even declare an interface method `synchronized`. The underlying reason: **functional behaviour composes; concurrency does not.**

(c) Code-level, ship tonight: put a timeout on the remote call and take the remote call out of the critical section — synchronise only the map read and the map write, never the fetch. (A dedicated bounded thread pool returning a future for the fetch is the fuller version.) **Architectural:** a bulkhead — give this feature its own bounded thread pool or its own instances — plus a circuit breaker around the partner API so you stop calling it at all when it's sick, with a fallback of “availability unknown”. Ship the timeout tonight; the architecture change wants a design conversation and a stakeholder decision about what the page shows when availability is unknown.

PART 3 · Q1 — SEARCH RESULTS IN THE SESSION

(a) Three, in this order: **Users** (session memory — a large object held for the full session timeout, most of it never read again, and the "dead time" after the last request is pure waste); then a memory leak in effect, which surfaces as **Slow Responses** as the collector works harder and harder; then a **Chain Reaction**, because every instance runs the same code with the same defect, and each death pushes its traffic onto peers that then exhaust memory faster. The accelerating crash intervals are the tell. If the result sets grew because customer history grew, you can fairly add **Unbounded Result Sets** as the root cause — and note that two months of production data is exactly the timescale on which that appears.

(b) Cheapest: **stop putting the result set in the session** — re-query the search engine for each page of results. Keeping an entire set of search results in the session for pagination is the textbook version of this mistake. If you must keep something, wrap it in a weak/soft reference so the collector can reclaim it, and make every accessor handle `null`.

(c) Class-eliminating: **pagination in the service contract** — first-item and count parameters going in, an approximate total coming back — plus a hard `LIMIT` on the query, plus production-sized data in the performance environment. Cost: a contract change every caller must adopt, callers must now loop, and "approximately how many results" is a real product decision rather than a free number.

PART 3 · Q2 — HOURLY PULLS, MIDNIGHT JOBS, TWITCHY AUTOSCALER

(a) A **Dogpile** (every agent pulling on the hour, every cache warming at midnight, and the fleet restarting with cold caches all at once), and a **Force Multiplier** (an autoscaler sampling on a one-second horizon while its actions take five minutes to land).

(b) The autoscaler is the **control-plane** problem, and it differs in kind: it isn't a bug and it isn't excess demand. The automation senses current state, compares it to a computed desired state, and acts — correctly, by its own logic — on a belief that is either stale or unmeasurable. That's what makes it dangerous: it will be working exactly as designed while it destroys you, and it'll finish before a human notices. The dogpile, by contrast, is ordinary demand arriving in the wrong *shape*.

(c) For the dogpile: **randomised clock slew** on the config agent and jitter on every scheduled job, so the pull spreads across the hour rather than landing on it; warm caches before an instance accepts traffic. That's the **Dogpile** counter, and jittered exponential backoff is its retry-side twin. For the autoscaler: a **Governor** — asymmetric (scale up readily, scale down slowly), stateful and time-aware, with a **deceleration zone** that accounts for the five-minute lag so it doesn't start three hundred instances because the load persisted for three hundred seconds, and an alert when it engages. Bonus marks for adding a financial ceiling and the >80%-unavailable heuristic.

PART 4 · Q1 — THREE DEPENDENCIES AND A MARKETING EMAIL

(a) (i) Inventory service, which you own: Handshaking — a health-check endpoint the caller respects, because you control both ends and cooperative throttling beats guessing. Back it with a timeout. **(ii) Third-party tax API, rate-limited, no health endpoint: Circuit Breaker**, explicitly because it *can't* handshake — you can't ask politely, so you make the call, track whether it works, and stop calling when it doesn't. Trip on fault density, and decide with the business what checkout does when it's open. **(iii) Slow internal fraud model: Timeouts** derived from your checkout SLA, plus a defined fallback (queue for asynchronous review, or accept-and-flag) — this is where **Decoupling Middleware** earns its keep if you can restructure.

(b) For your own inbound traffic: **Shed Load** (with **Fail Fast** as its per-request form). It's not the same pattern as (ii) because the two sit on opposite sides of the call. A circuit breaker is a *caller-side* decision to stop making outbound calls; shed load is a *provider-side* decision to refuse inbound work. Note also that shed load is right here specifically because the demand is uncontrolled and effectively infinite — inside your own boundary, with a finite set of known consumers, **Back Pressure** would be the more efficient choice.

(c) The conversation with **marketing**, last week. Self-denial attacks are the most preventable class precisely because somebody in your own building already knew. Ask for the send time and volume, get the deep link removed so the CDN isn't bypassed, get the mail sent in waves, put a static landing page behind the first click, and pre-scale before the send rather than trusting autoscaling to catch up.

PART 4 · Q2 — THE SHARED-STATE CIRCUIT BREAKER

(a) On where the state lives: a circuit breaker should be built at the scope of a **single process**. Sharing state through Redis introduces another out-of-process call — "that means the safety mechanism would introduce a new failure mode." Your protection now has a dependency that can itself be slow, full, or unreachable, and it fails at exactly the moment everything else is failing. The cost of not sharing is small and known: several instances each independently discover the provider is down. **On how you count:** a simple total is nearly meaningless. "There's a world of difference between observing five faults spread evenly over five hours versus five faults in the last thirty seconds." You want fault **density**, not a running total, or the breaker will eventually trip on ancient history.

(b) A **Leaky Bucket**: a counter incremented on every observed fault, with a background thread or timer decrementing it periodically toward zero. If the count exceeds the threshold, faults are genuinely arriving quickly. Consider tracking fault *types* separately too — a lower threshold for “timeout calling remote system” than for “connection refused”.

(c) Keep the state local and make the **observability** global. Log every state change, expose current state for querying, and ship those events to your monitoring system; then chart the aggregate. State-change frequency is itself a valuable metric — a leading indicator of problems elsewhere in the enterprise. You get the global picture without putting a network call inside your safety mechanism. Add a manual trip/reset control for Operations while you're there.

HOW TO SCORE YOURSELF

Two marks per question, both quick. **Completeness:** did you name every element the answer names, and did you get the *direction* right (caller vs provider, sideways vs upward)? Missing the direction is the failure that matters. **Phrasing:** could you have said your answer out loud in a design review without hand-waving? If you wrote “it broke” where the key says “blocked threads carried a cascading failure across the layer boundary”, the concept is there but the vocabulary isn't — and the vocabulary is half the point.

SECTION 10

Spaced-review tracker

Review at **day 1, 3, 7, 16, 35**. Write the date in the box when you pass. On a fail, reset that row to a 1-day interval and start again — the reset is the whole point, not a punishment.

ITEM	DAY 1	DAY 3	DAY 7	DAY 16	DAY 35	FAILS
1 · Fast vs slow failure — and why slow is worse						
2 · The 5 a.m. problem: mechanism, clue, fix						
3 · Chain Reaction vs Cascading Failure — direction and carrier						
4 · Blocked Threads — why monitoring says “healthy”						
5 · Slow Responses — why they *increase* load						
6 · Scaling Effects & Unbalanced Capacities — why QA can't find them						
7 · Self-Denial · Dogpile · Force Multiplier — the three self-inflicted ones						
8 · Unbounded Result Sets — why it's a handshaking failure						
9 · Timeouts vs Fail Fast — outbound vs inbound						
10 · Circuit Breaker — three states, fault density, single process						
11 · Bulkheads — granularities, and what they don't protect						
12 · Steady State — the three sludges, and “no fiddling”						
13 · Let It Crash — the four preconditions						
14 · Shed Load vs Back Pressure — edge vs inside						
15 · Governor — asymmetric, stateful, U-shaped curve						
16 · Test Harness vs mock vs integration environment						
17 · Decoupling Middleware — the coupling spectrum, and why it's irreversible						
18 · Master 2×2 — coupling × interactive complexity						

Final quiz — no notes, twelve minutes, write the answers

1. You have one change you're allowed to make to a legacy service with forty outbound calls and no timeouts anywhere. What do you change, and which four antipatterns does that single change address?
2. Name the direction of travel and the carrying mechanism for: a chain reaction, a cascading failure, a slow-response amplification loop.
3. Your monitoring reports that 94% of your fleet is unavailable. State your first hypothesis and your reason for it.
4. Give the one-line difference between Shed Load and Back Pressure, and name the property of your consumer population that decides which you use.
5. A teammate says “we'll add data purging six months after launch.” Give the two-sentence rebuttal, naming the pattern.
6. Why is a bulkhead no help to the callers of the partition that fails, and what is?
7. Name the four conditions that must hold before “let it crash” is a responsible strategy, and one system type where it is simply wrong.
8. A query “can only ever return a handful of rows.” Give the two-sentence reply, including the sentence about which numbers are sensible.
9. Explain in one sentence why sharing circuit-breaker state across processes is a bad trade.
10. Where would you rather be on the coupling × complexity matrix, which direction can you realistically move, and name three things that move you there.